Визуализация данных на языке R.

Часть. 1. Визуализация временных рядов (визуализация темпоральных данных)

Задание 1. Создание гистограммы диаграммы на языке R
с последующей обработкой в графическом редакторе

Будем визуализировать результаты Нейтановского турнира по поеданию хот-догов за последние три десятилетия.

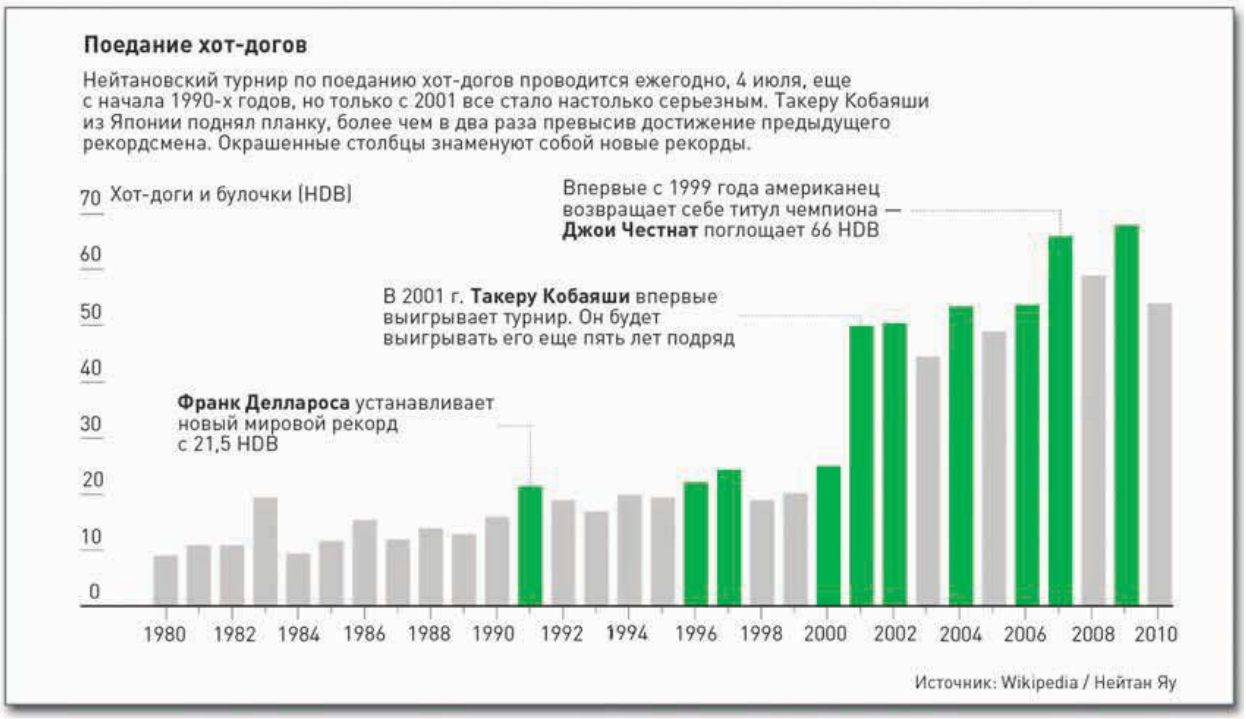


Рисунок 1.1. Вид готовой диаграммы

Работа состоит из нескольких этапов:

1. Создадите базовый вариант столбцовой диаграммы на языке R.
2. Обработайте полученный результата в графическом редакторе.

Нейтановский турнир по поеданию хот-догов — это ежегодное мероприятие, которое проходит 4 июля. В Соединенных Штатах это День независимости. Соревнование стало настолько популярным, что его даже показывают по телевидению на канале ESPN. С конца 1990-х годов примерно за 15 минут победители съедали от 10 до 20 хот-догов и булочек (HDB). Однако в 2001 году Такеру Кобаяши (Takeru Kobayashi), профессиональный едок из Японии, наголову разбил конкурентов, съев 50 HDB. Это в два с лишним раза больше того количества, которое кто-либо в мире съедал до него.

1. Файл csv формата во вложении к заданию: hot-dog-contest-winners.csv.
2. Чтобы загрузить данные в R, используйте команду read.csv().

а) Вы можете обратиться к файлу по URL. Для этого введите в код на R следующую строку:
hotdogs <-read.csv("http://datasets.flowingdata.com/hot-dog-contest-winners.csv", sep=",", header=TRUE).

Командой **read.csv()** вы загружаете данные. У нее есть три аргумента. Первый — это **местоположение** ваших **данных**. В приведенном примере это URL. Второй аргумент, **sep**, определяет, какой **знак разделяет колонки** в файле с данными. В нашем случае это файл с **разделителями-запятыми**, а потому указывается запятая. Если бы это был файл, в котором разделителем является **символ табуляции**, тогда бы вам пришлось использовать не запятую, а **\t**. Последний аргумент — **header** — сообщает R, что у файла с данными есть **шапка**, которая содержит **названия всех колонок**. Первая колонка — это год, вторая — имя победителя, третья — количество съеденных хот-догов и булочек, четвертая — страна, которую представлял победитель, новый рекорд (New record). Если в соответствующем году мировой рекорд оказался побит, в ячейке будет стоять 1, в противном случае — 0.

Вот как будет выглядеть начало таблицы данных, если вы наберете **hotdogs**:

	Year	Winner	Dogs.eaten	Country	New.record
1	1980	Paul Siederman & Joe Baldini	9.10	United States	0
2	1981	Thomas DeBerry	11.00	United States	0
3	1982	Steven Abrams	11.00	United States	0
4	1983	Luis Llamas	19.50	Mexico	1
5	1984	Birgit Felden	9.50	Germany	0

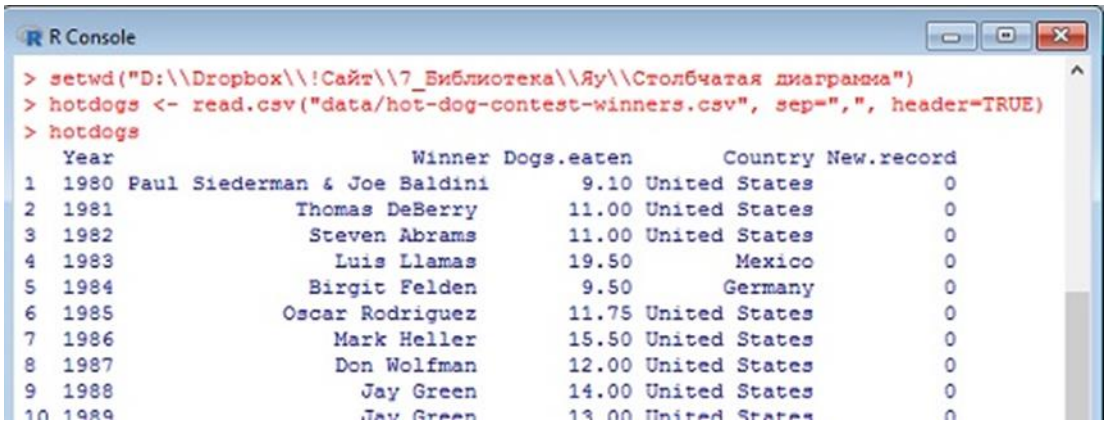


Рисунок 1.2. Таблица с исходными данными, загруженная в R

Пробелы в названиях колонок заменяются точками. Вместо Dogs eaten теперь имеем Dogs.eaten. Для обращения к конкретной колонке данных вам нужно использовать название таблицы, за которым следует знак доллара $ и название колонки. Например, если вы хотите обратиться к Dogs.eaten, необходимо ввести следующее: hotdogs$Dogs.eaten.

б) Если же вы хотите загрузить данные локально, используйте главное меню и поместите свой рабочий каталог в R в ту же директорию, в которой находится и ваш файл с данными. Или же вы можете воспользоваться функцией setwd(), которая позволяет изменить свой рабочий каталог.
Пример:

```
# Change directory
setwd("D:\\Dropbox\\!Сайт\\Библиотека\\Инфографика\\Столбчатая диаграмма")

# Load data
hotdogs <- read.csv("data/hot-dog-contest-winners.csv", sep=",", header=TRUE)
```

Попробуйте оба способа загрузки данных.

3. Теперь, когда данные уже в R, вы можете сразу же перейти к вычерчиванию диаграммы с помощью команды barplot().

```
barplot(hotdogs$Dogs.eaten)
```

Таким образом вы отдаете распоряжение R начертить диаграмму на основе данных из колонки Dogs.eaten. В результате вы получите столбцовую диаграмму, представленную на рисунке 3.

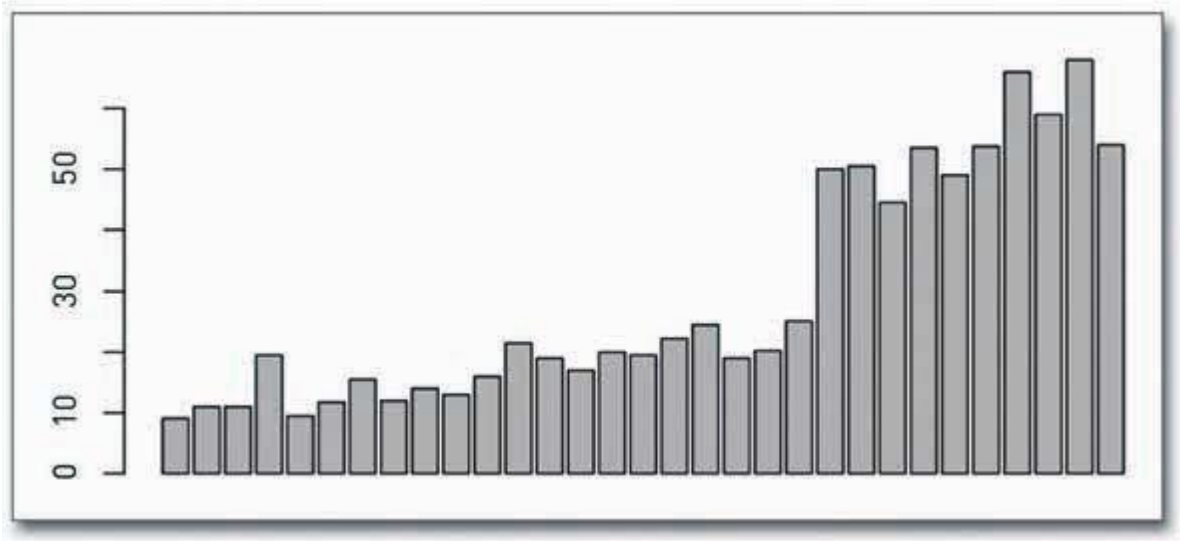


Рисунок 1.3. Столбцовая диаграмма, построенная в R

4. Используйте аргумент names.arg в barplot(), чтобы дать названия столбцам. В данном случае это год проведения каждого соревнования.

```
barplot(hotdogs$Dogs.eaten, names.arg=hotdogs$Year)
```

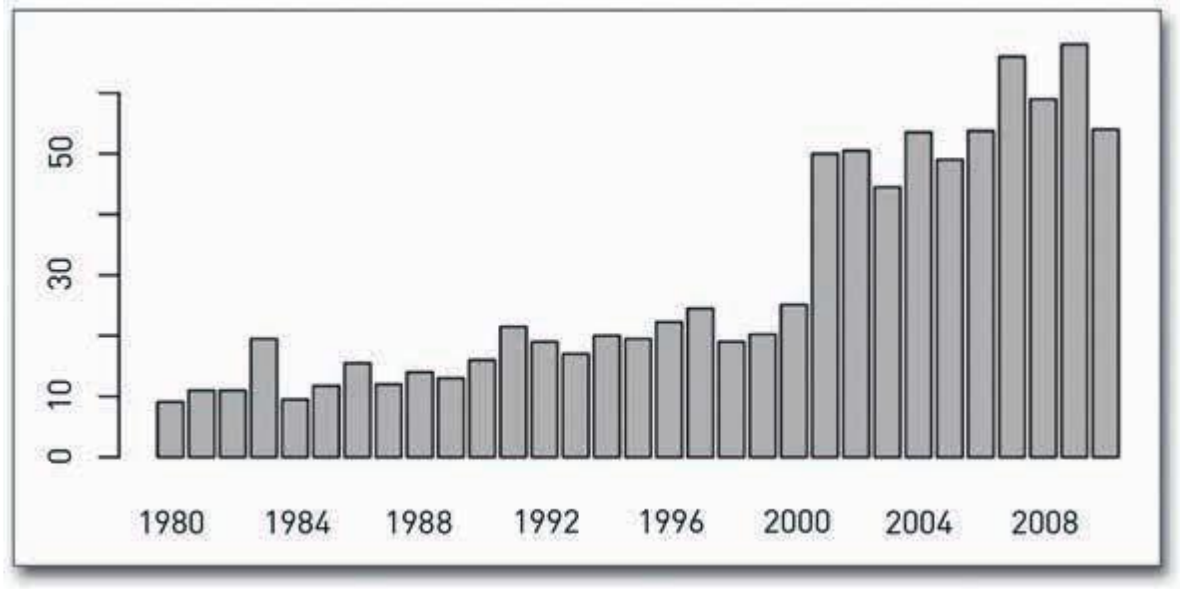


Рисунок 1.4. Столбцовая диаграмма, построенная в R с указанием названий стлобцов

5. Теперь давайте изменим подписи к осям, рамки и добавим цвета. Для этого будем использовать другие аргументы функции построения графиков в R:

```
barplot (hotdogs$Dogs.eaten, names.arg=hotdogs$Year, col="red", border=NA, xlab="Годы", ylab="Число съеденных хот-догов")
```

Аргумент col служит для задания цвета (возможные варианты перечислены в документации к R) или шестнадцатеричного числа, такого как #821122. В данном случае был выбран вариант без границ — NA (это логическая константа, означающая «без значения»). Кроме того, были добавлены подписи к осям X и Y: «Годы» и «Число съеденных хот-догов» соответственно.

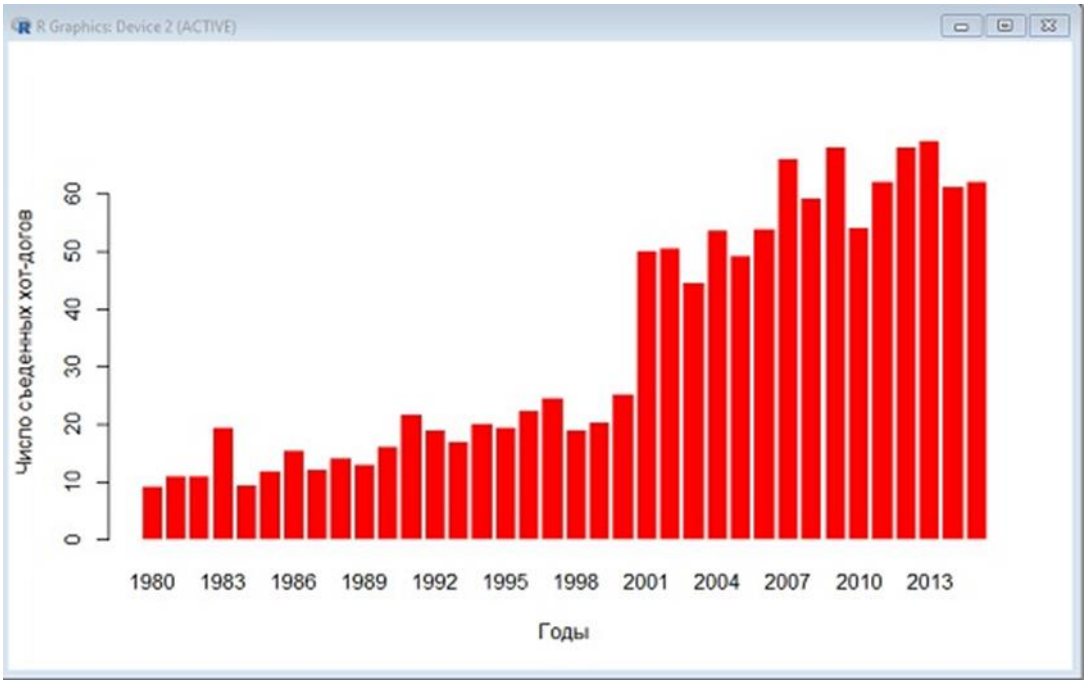


Рисунок 1.5. Столбчатая диаграмма с окрашенными столбиками и подписями к осям

6. Теперь перечислим в barplot() множество цветов, раскрасив каждый столбик по разному. Допустим, вам нужно выделить годы, когда Соединенные Штаты выигрывали соревнование. Эти годы вы можете окрасить в темно-красный цвет (#821122), а все остальные — в светло-серый, как это показано на рисунке 6.

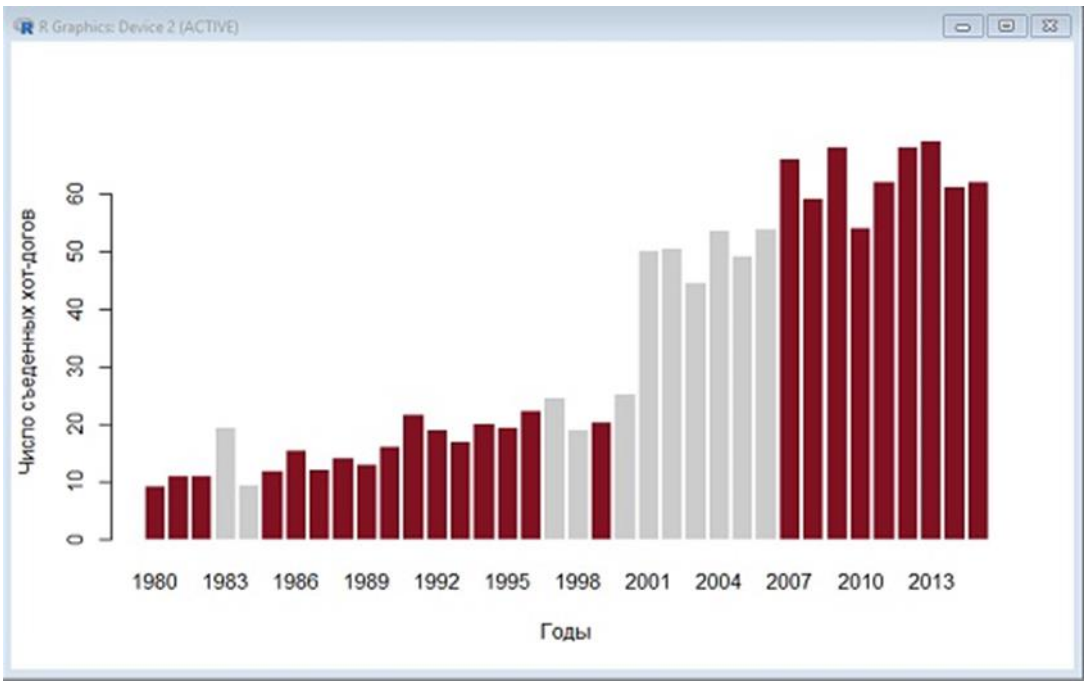


Рисунок 1.6. Столбчатая диаграмма с индивидуально окрашенными столбиками

Первая строка создает пустой вектор под именем fill_colors. В R для создания векторов используется команда c(). Следующая строка запускает цикл for для индекса i от 1 до числа строк в вашей таблице данных hotdogs. Вы можете взять колонку Country и вычислить ее длину (количество элемнетов). Если бы вы использовали length() только с hotdogs, тогда вы бы получили число колонок (5), но вас интересует количество строк (36). Для каждого года с 1980 по 2015 существует отдельная строка, а значит, цикл выполнит код внутри скобок 36 раз, и с каждым разом индекс i будет увеличиваться на 1.

При первой итерации, когда i равняется 1, производится проверка того, является ли страна в первом ряду (то есть страна победителя 1980 года) США. Если да, то с помощью fill_colors присваивается цвет

#821122 (это и есть тот самый оттенок темно-красного в шестнадцатеричной форме). В противном случае присваивается #cccccc (серый). Чтобы увидеть, чем все закончится, введите в консоль fill_colors. Это тот самый вектор цветов, который вам нужен, смотрите рисунок 7.

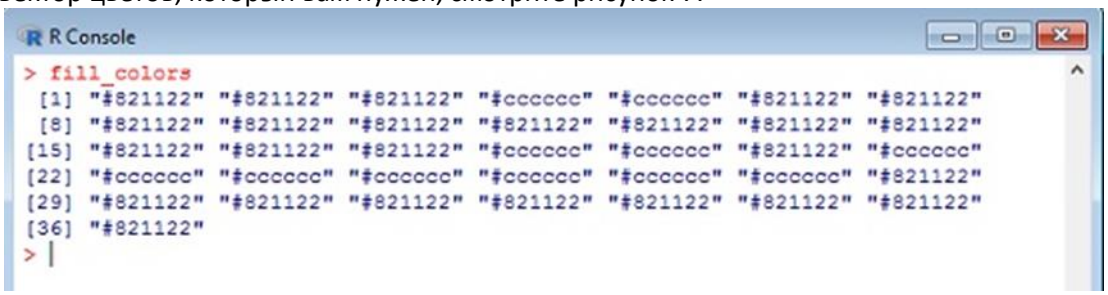


Рисунок 1.7. Вектор цветов fill_colors

Передайте вектор fill_colors в аргумент col для barplot():

barplot(hotdogs$Dogs.eaten, names.arg=hotdogs$Year, col=fill_colors, border=NA, xlab="Годы", ylab="Число съеденных хот-догов").

Код остался тем же, что и прежде, за тем исключением, что в аргументе col вы используете fill_colors, а не "red".

7. В окончательном варианте столбчатой диаграммы надо выделить, однако, не те годы, когда побеждали Соединенные Штаты, а те, когда устанавливался новый мировой рекорд. Информация о новых рекордах в таблице данных содержится в колонке New.record, и если там стоит 1, то столбец должен быть темно-красным, в противном случае — серым (рисунок 8):

```
# Highlight record years with color
fill_colors <- c()
for ( i in 1:length(hotdogs$New.record) ) {
	if (hotdogs$New.record[i] == 1) {
		fill_colors <- c(fill_colors, "#821122")
	} else {
		fill_colors <- c(fill_colors, "#cccccc")
	}
}
barplot(hotdogs$Dogs.eaten, names.arg=hotdogs$Year, col=fill_colors, border=NA, xlab="Годы", ylab="Число съеденных хот-догов")
```

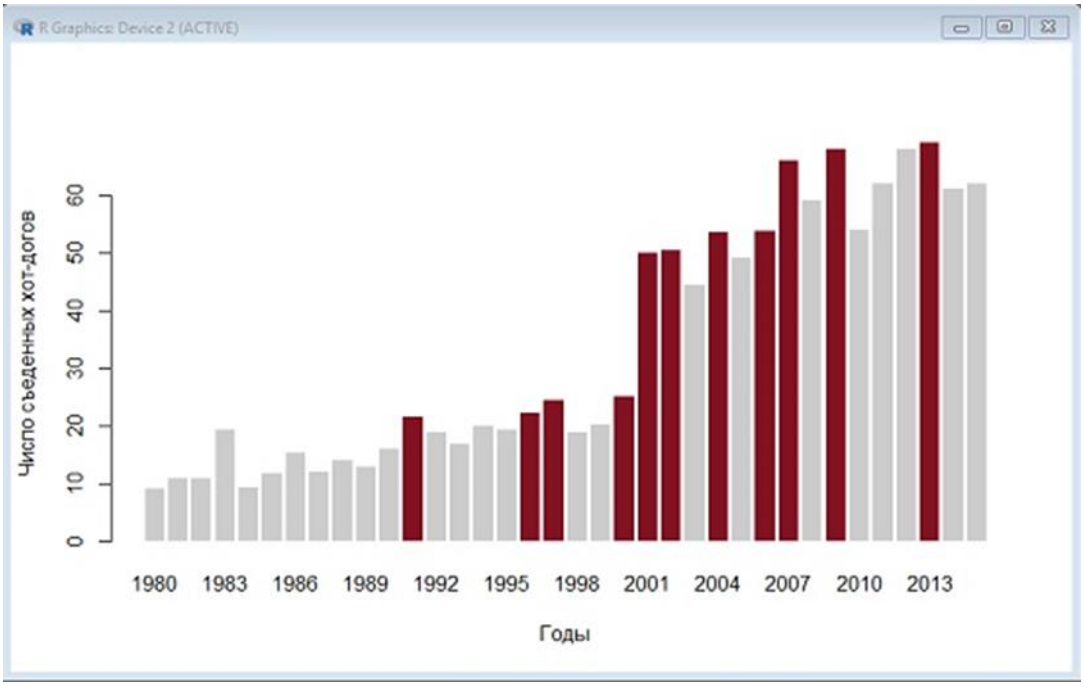


Рисунок 1.8. Столбчатая диаграмма с подкрашенными рекордными значениями

8. На последнем этапе вы можете поэкспериментировать и с другими аргументами barplot(), например изменить расстояние между столбцами или добавить заголовок (рис. 9). Обратите внимание, что интервал между столбцами стал больше, чем прежде, и появился заголовок.

barplot(hotdogs$Dogs.eaten, names.arg=hotdogs$Year, col=fill_colors, border=NA, space=0.3, xlab="Годы", ylab="Число съеденных хот-догов", main="Результаты Найтоновского турнира по поеданию хот-догов, 1980-2015")



Рисунок 1.9. Столбчатая диаграмма с измененными интервалами между столбцами и заголовком

Можно сохранить графический файл в формате pdf. Для этого выделите окно графика, и пройдите в консоли по меню Файл – > Сохранить как –> PDF. Однако, если подписи содержат текст на русском языке, они будут отображаться некорректно. Сохраните файл с помощью следующего кода:

pdf("Nathan.pdf", family="NimbusSan")

barplot(hotdogs$Dogs.eaten, names.arg=hotdogs$Year, col=fill_colors, border=NA, space=0.5, xlab="Годы", ylab="Число съеденных хот-догов", main="Результаты Найтоновского турнира по поеданию хот-догов")

dev.off()

embedFonts("Nathan.pdf")

Чтобы просмотреть документацию по любой функции R, просто наберите знак вопроса и за ним название функции. Например, чтобы получить информацию о barplot(), просто наберите ?barplot. Вы перейдете на соответствующую страницу справки в Интернете. Справка обычно сопровождается примерами, которые бывают исключительно полезными.

К заданию 1. Обработка готовой диаграммы в графическом редакторе

Посмотрите на диаграмму с точки зрения сторителлинга (подробнее читайте статью из книги *Мартин Сайкс и др. «От слайдов к историям. Пошаговая методика создания убеждающих презентаций»*). Представьте себе, что вы видите рис. 9 впервые. Вы — читатель и случайно наткнулись на эту диаграмму. Какую информацию вы можете почерпнуть из нее? Вы знаете, что на диаграмме отображено поедание хот-догов по годам. Значит ли это, что речь идет о пищевых привычках конкретного человека? Для одного человека хот-догов явно многовато. Может, это корм для животных? А остатки отдают птичкам? Или здесь представлено количество хот-догов, поедаемых людьми за год в среднем? И почему столбцы окрашены?

Поскольку вы — тот самый человек, который делал эту диаграмму, вам известен контекст, стоящий за каждым числом. Но читатели ничего не знают, а потому вам следует объяснить им, что здесь происходит. Хороший дизайн помогает читателям лучше вникнуть в историю. Программа Illustrator позволяет вручную манипулировать каждым из элементов графики. Вы можете задать шрифты, добавить примечания, видоизменить оси, отредактировать цвета и вообще сделать все, что подскажет вам ваша фантазия.

С сайта Adobe можно установить бесплатную пробную версию Illustrator (в качестве альтернативы можете попробовать Inkscape, распространяемую бесплатно). Откройте в Illustrator PDF-файл с вашей диаграммой. Обратите внимание на окно инструментов (рис. 10). Если вы не видите этого окна, то в пункте главного меню *Окно* выберите команду *Инструменты*.



Рисунок 1.10. Окно инструментов Adobe Illustrator

Черная стрелка — это инструмент *Выделение*. Выберите его, и указатель вашей мыши превратится в черную стрелку. Щелкните ею по любому объекту. Появится выделение. В Illustrator это называется обтравочной маской. Она может пригодиться в самых разных ситуациях, но сейчас она вас не интересует, так что нажмите клавишу Delete на клавиатуре и избавьтесь от маски. Если в результате будет удалена вся диаграмма или ее элемент, отмените правку (Ctrl+Z) и используйте для выделения обтравочной маски инструмент *Прямое выделение*, который обозначен серой стрелкой.

Измените шрифт. Снова, используя инструмент *Выделение*, щелкните по тексту, чтобы отметить, что именно вы хотите изменить. Вызовите меню *Символ* (*Текст*), набрав Ctrl+T. Поменяйте шрифт на любой другой.

Выделите числовые подписи оси ординат. Перейдите в меню *Объект* и выберите *Трансформировать –> Трансформировать каждый*. Измените угол вращения на –90° (рис. 11). Подписи выстроятся по горизонтали (у меня команда работала не так как описано, и подписи выстраивались в одну горизонтальную линию, так что пришлось их растаскивать вручную). Поднимите подписи (только подписи, не линии разметки) вверх и вправо чтобы они оказались над метками на оси, а не слева от них.

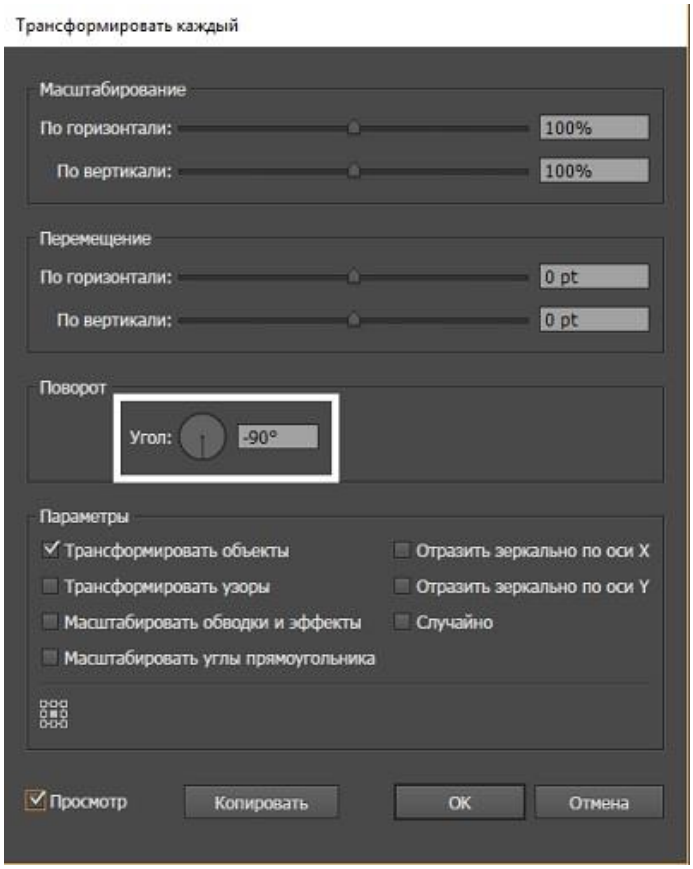


Рисунок 1.11. Окно *Трансформировать каждый*

Удалите вертикальную линию оси значений. Она не несет никакой полезной информации. Если вы выберете вертикальную линию инструментом *Выделение*, подписи также окажутся выделенными. Так происходит, потому что они являются частью группы. Чтобы выбрать одну линию, используйте инструмент *Прямое выделение*. После того как линия окажется выделенной, нажмите клавишу Delete, и она исчезнет. Добавьте полужирную линию горизонтальной оси. Обратите внимание на то, что в окончательном варианте диаграммы мы имеем только штриховые метки (рис. 12). О том, какие элементы диаграммы только мешают восприятию см. *Принцип Эдварда Тафти минимизации количества элементов диаграммы*.

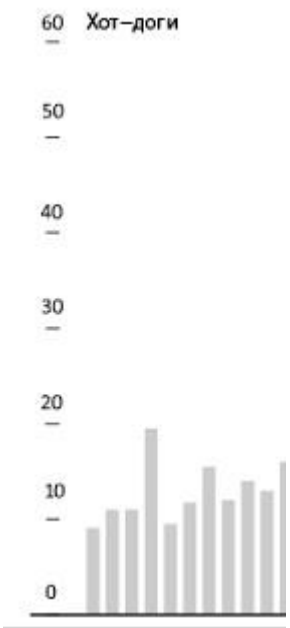


Рисунок 1.12. Ось значений преобразована

Существует множество способов создания линий разметки, но мы сейчас остановимся только на одном из них. Выберите инструмент *Перо* из окна инструментов, а затем установите стиль ваших линий в окне *Обводка*. Задайте толщину штриха 0,25 pt и убедитесь, что флажок *Пунктирная линия* не активен.

Чтобы начертить линию, щелкните мышью в том месте, где вы хотите расположить первую точку линии (или штриха разметки, как в данном случае), а затем щелкните там, где хотите поставить вторую точку. Если во время второго щелчка вы будете держать клавишу Shift, то автоматически получите прямую линию. Пока у вас есть одна метка. Но вам нужны еще 30 — по метке на каждый год. Вы можете проставить их все вручную, но есть способ лучше. Нажмите клавишу Alt. Таким образом будет создана копия, и у вас окажутся две метки. Далее нажмите Ctrl + D. Сделав это, вы продублируете новую метку, причем с таким же интервалом от предыдущей, как между первыми двумя. Нажимайте Ctrl + D до тех пор, пока не получите необходимое количество меток. Наконец, вам нужно расположить все метки правильно. Сдвиньте последнюю из них так, чтобы она оказалась в центре под последним столбцом. Первая метка и так уже должна находиться в центре под первым столбцом. Теперь выделите все метки и выберите в окне *Выравнивание* вариант *По горизонтали по центру* (рис. 13). После этого все метки распределятся на равных расстояниях между двумя крайними. Если вы захотите, то с помощью инструмента для *Выделение* можете выбрать каждую вторую метку и изменить ее размер по вертикали, сделав ее короче. Так с первого взгляда будет понятно, что более длинные метки относятся к сопроводительным подписям с годами. (Надо отметить, что и здесь мне не удалось воспользоваться рекомендациями автора, так что риски установил вручную, а затем их выровнял по вертикали.)

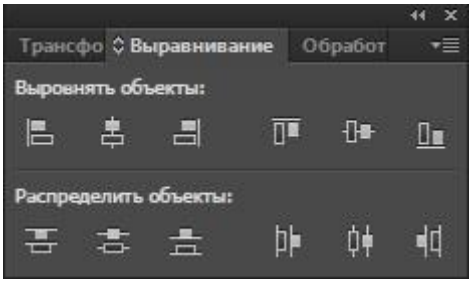


Рис. 1.13. Окно *Выравнивание*

Чтобы изменить цвет заливки с красного на зеленый, вы можете вернуться к инструменту *Прямого выделения* и выбрать по очереди все красные прямоугольники. Поскольку их относительно мало, это можно выполнить довольно быстро. Но что делать, если таких элементов много? Тогда лучше будет поступить следующим образом: щелкнуть по одному-единственному красному столбцу, а затем выбрать *Выделение –> По общему признаку –> С одинаковым цветом заливки*. Теперь поменяйте цвет на тот, который вам больше нравится, через окно Цвет. Здесь вы можете перекрасить и границы, и заливку — однако сейчас вам нужно изменить только заливку (рис. 14).

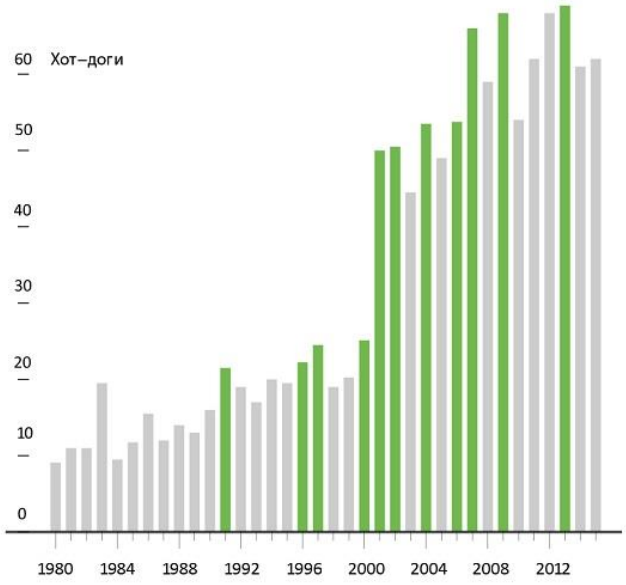


Рис. 1.14. Изменение цвета элементов диаграммы

Используйте инструмент *Текст*, который находится в окне *Инструменты*, чтобы добавить к диаграмме текстовые окошки. Это ваш шанс объяснить читателям, что именно они видят перед собой, глядя на диаграмму, и прояснить для них любые непонятные моменты. Выберите шрифт, который кажется вам подходящим, — причем установите другой кегль и гарнитуру, чтобы подписи отличались от остальных элементов диаграммы, таких как названия осей.

В случае с этой сверхважной хот-договой диаграммой особо стоит выделить первый с 1980 года рекорд, а также период, когда доминировал Такеру Кобаяши, а также сегодняшнее лидерство Джои Честната. Добавьте также заголовок и вводный абзац, объясняющий суть диаграммы. И вот что еще немаловажно: не забудьте указать источник данных. Без этого не будет никакого способа определить, насколько ваша диаграмма верна. Сложите все вместе, и вы получите окончательный вариант диаграммы, который представлен в самом начале.

Задание 2. Построение штабельной диаграммы в R

Алгоритм построения любой диаграммы на языке R очень прост:

1. Загрузите данные.

2. Убедитесь, что данные правильно отформатированы.

3. Используйте функцию R, чтобы создать диаграмму.

Будем строить штабельную диаграмму, используя данные Нейтановского турнира по поеданию хот-догов, файл можно скачать по ссылке: http://datasets.flowingdata.com/hot-dog-places.csv. Получить хотим картинку, как на рисунке 1.

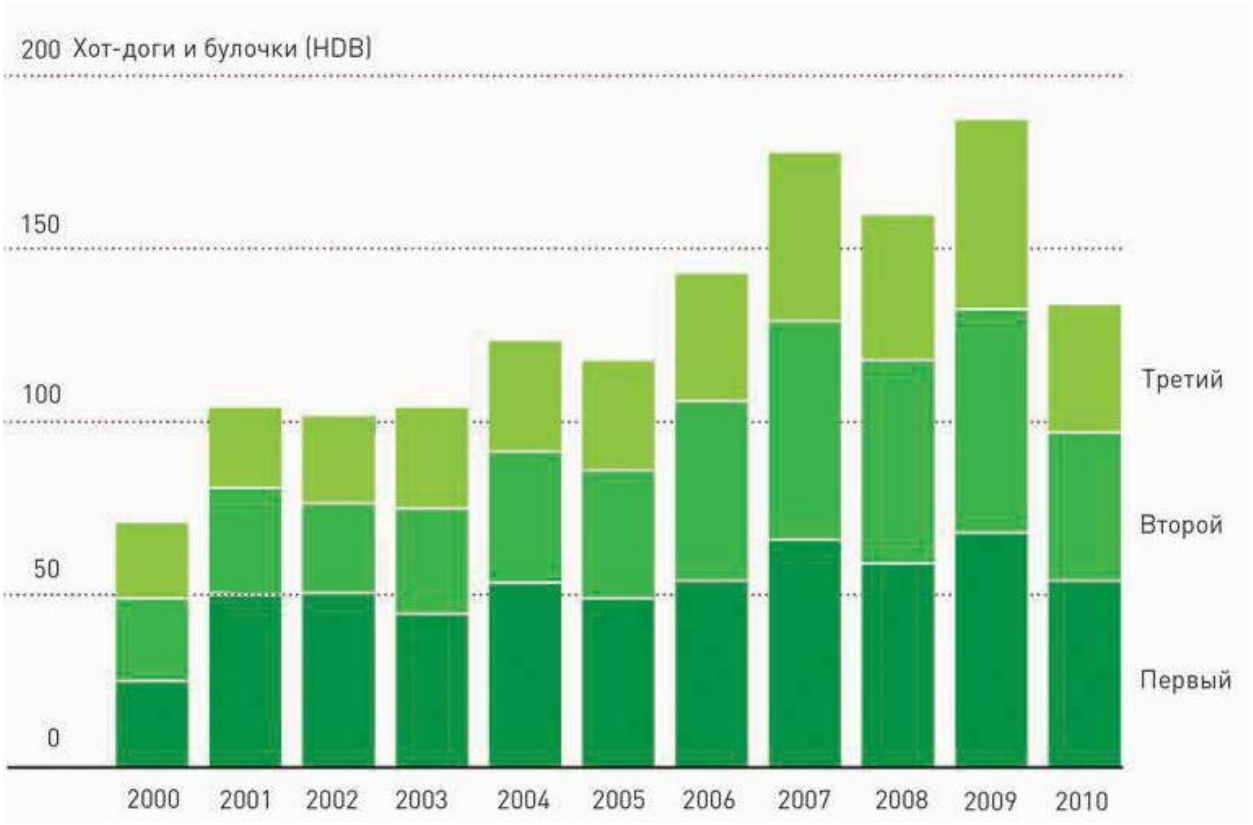


Рисунок 2.1. Штабельная диаграмма

1. Загрузка данных в R.

```
hot_dog_places <-

read.csv('http://datasets.flowingdata.com/hot-dog-places.csv', sep=",", header=TRUE)
```

Наберите hot_dog_places, чтобы посмотреть загруженные данные, рисунок 2. Каждая колонка показывает результаты за конкретный год, а каждая строчка — за места от первого по третье.

2. Обработка и форматирование данных.

Обратите внимание на то, что все названия колонок предваряются знаками «X». Они были добавлены по умолчанию в тот момент, когда вы загружали данные, потому что названия в шапке состоят из чисел. R это не нравится, вот он и добавляет буквы, чтобы сделать названия более похожими на слова или, говоря компьютерным языком, превратить их в строку. Названия в шапке

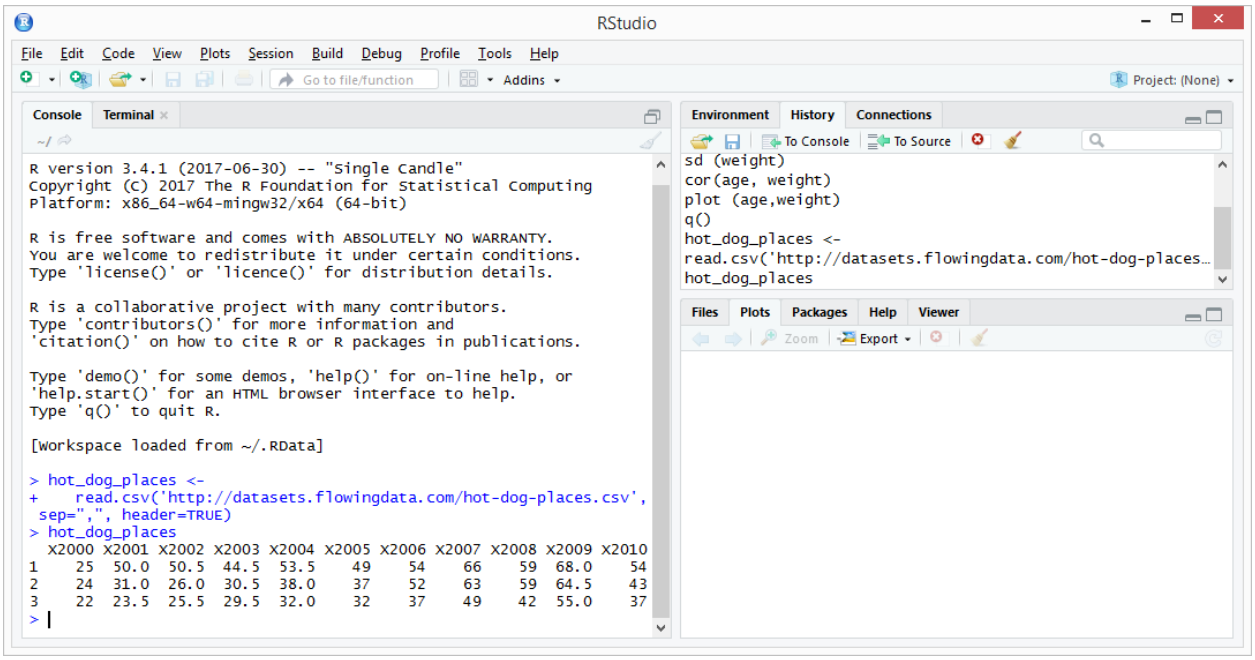


Рисунок 2.2. Загрузка данных в R

понадобятся вам для подписей в штабельной столбцовой диаграмме, а потому лучше вернуть им первоначальный вид:

names(hot_dog_places) <- c("2000", "2001", "2002", "2003", "2004", "2005", "2006", "2007", "2008", "2009", "2010").

Используйте кавычки и оградите ими каждый год, чтобы уточнить, что это именно строка. Наберите снова hot_dog_places, теперь шапка должна появиться в нормальном виде — с годами. Проверьте, как теперь будут отображаться данные.

3. Построение диаграммы.

Как и в предыдущем примере, вы будете использовать функцию barplot(), но данные у вас теперь в другом формате. Чтобы передать все верхние данные в barplot(), вам необходимо конвертировать hot_dog_places в матрицу. Сейчас это таблица данных. В R таблица и матрица — разные структуры, но сейчас отличия не так важны. Важнее знать, как конвертировать таблицу данных в матрицу:

hot_dog_matrix <- as.matrix(hot_dog_places)

Только что созданная матрица была сохранена как hot_dog_matrix. Теперь можно передавать ее в barplot().

barplot(hot_dog_matrix, border=NA, space=0.25, ylim=c(0, 200), xlab="Year", ylab="Hot dogs and buns (HDB) eaten", main="Hot Dog Eating Contest Results, 1980-2010")

Эти строки определяют, что вокруг столбцов не будет границ, что расстояние между столбцами должно составлять 0,25 от ширины одного столбца и что значения на оси ординат будут в пределах от 0 до 200. Здесь же задаются заголовок и названия осей. Результат представлен на рис.3

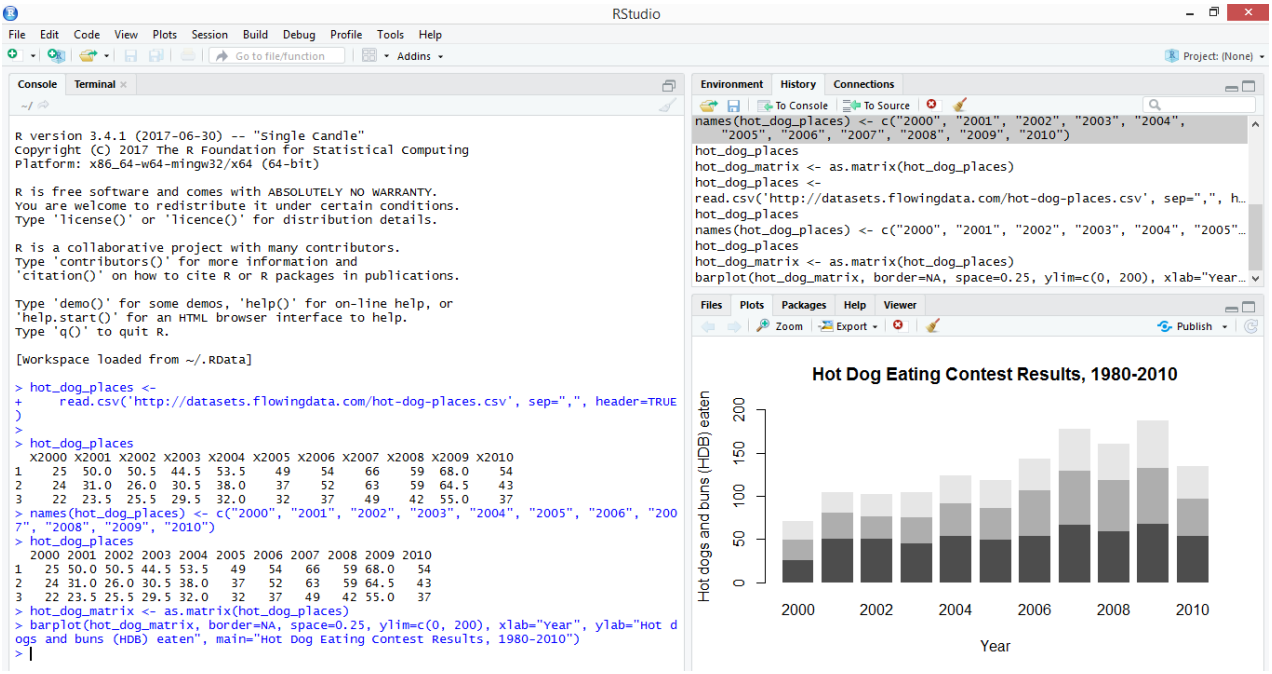


Рисунок 3. Штабельная диаграмма, построенная в R.

Теперь можно приступить к наведению красоты. Сохраните изображение как PDF и откройте его в программе Illustrator. Используйте те же самые инструменты, что и в предыдущем примере. Вы можете добавить немного текста с помощью инструмента Type (Текст), изменить шрифты, упростить вертикальную ось, отредактировать цвета, используя возможность выделять элементы с одинаковой заливкой. Получите изображение сходное с рисунком 1.

Задание 2.1 Построение точечного графика в R

Данный тип диаграмм известен больше как диаграмма рассеяния, и ее можно использовать для визуализации не только темпоральных данных. Нередко такую диаграмму применяют для демонстрации зависимости между двумя переменными.

Будем использовать данные подсчета подписчиков FlowingData из предыдущей лабораторной работы.

1. Загрузка данных.

```
subscribers <-read.csv("http://datasets.flowingdata.com/flowingdata_subscribers.csv", sep=",", header=TRUE)
```

Чтобы посмотреть первые пять рядов данных, введите:

```
subscribers[1:5,]
```

На экран выведется таблица:

```
    Date Subscribers Reach Item.Views  Hits
1 01-01-2010        25047  4627       9682 27225
2 01-02-2010        25204  1676       5434 28042
3 01-03-2010        25491  1485       6318 29824
4 01-04-2010        26503  6290      17238 48911
5 01-05-2010        26654  6544      16224 45521
```

У нас имеется пять колонок: с датой (Date), количеством подписчиков (Subscribers), охватом (Reach), просмотрами публикаций (Item.Views) и хитами (Hits). Нас интересуют только подписчики. Вы можете включить также и дату, но подсчет и без того ведется в хронологическом порядке, так что для нанесения количества подписчиков на диаграмму первая колонка вам не нужна. Чтобы нарисовать точки, введите приведенную ниже строчку, и в результате вы получите диаграмму, как на рис. 5.

plot(subscribers$Subscribers)

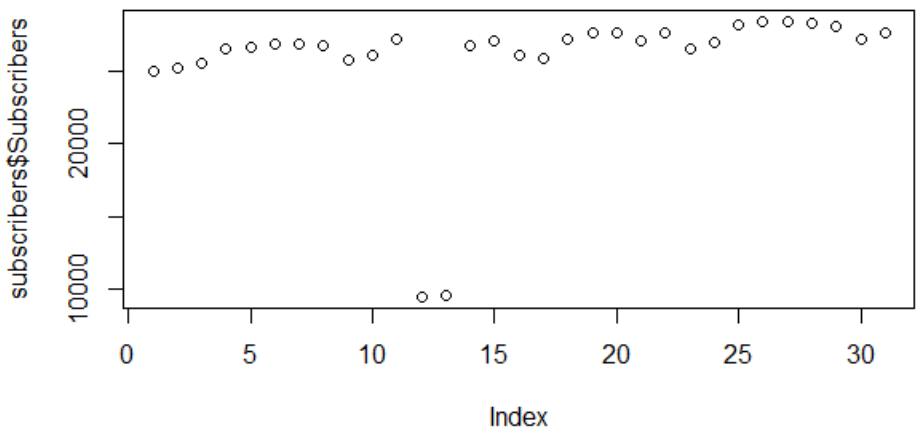


Рисунок 5. Диаграмма рассеяния.

Функция plot() на самом деле позволяет создавать несколько различных типов диаграмм, но по умолчанию рисуется именно диаграмма рассеяния. Сейчас вы использовали только данные о количестве подписчиков. Когда вы предоставляете функции plot() всего один массив данных, она решает, что этот массив содержит значения, и автоматически генерирует индекс для x-координат. А теперь давайте зададим конкретно, какой именно тип диаграммы нам нужно получить, и установим диапазон значений вертикальной оси от 0 до 30 000.

plot(subscribers$Subscribers, type="p", ylim=c(0, 30000))

Теперь диапазон значений вертикальной оси стал шире, как это и было задано в аргументе ylim, рисунок 6. Обратите внимание на аргумент type, указывающий, что нужно использовать точки. Если вы поменяете type на h, тогда R нарисует плотные вертикальные линии, рисунок 7

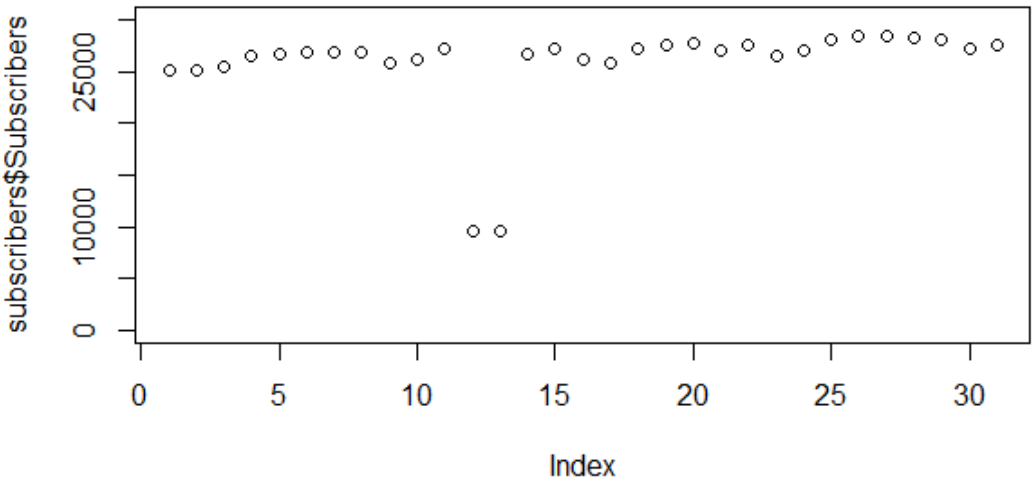


Рисунок 2.6. Точечная диаграмма в R с заданными предельными значениями оси Y

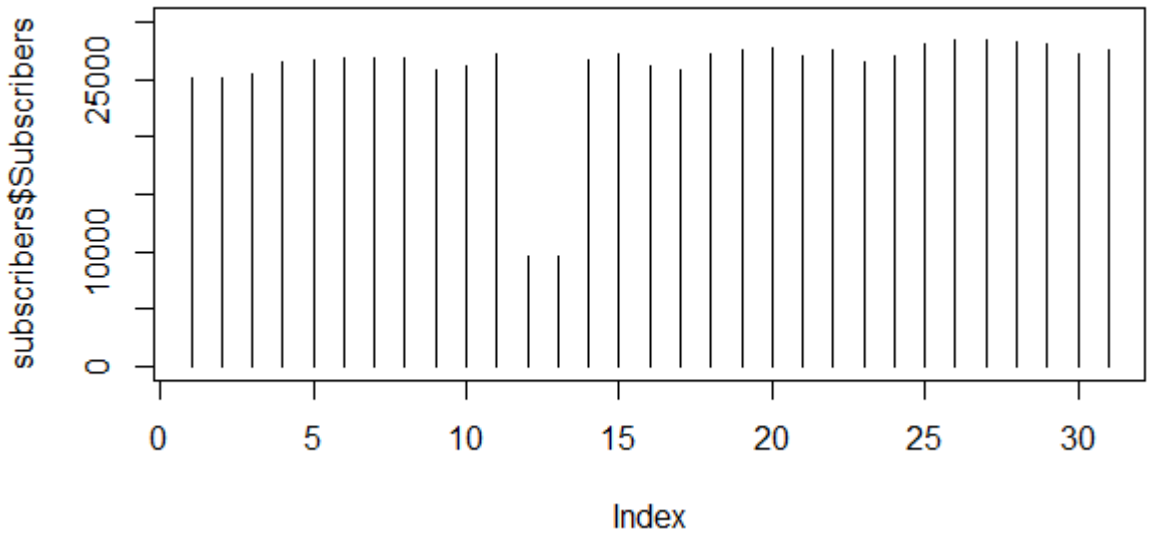


Рисунок 2.7. Линейчатая диаграмма.

Попробуем скомбинировать точки и линии, рисунок 8:

plot(subscribers$Subscribers, type="h", ylim=c(0, 30000), xlab="Day", ylab="Subscribers")

points(subscribers$Subscribers, pch=19, col="black")

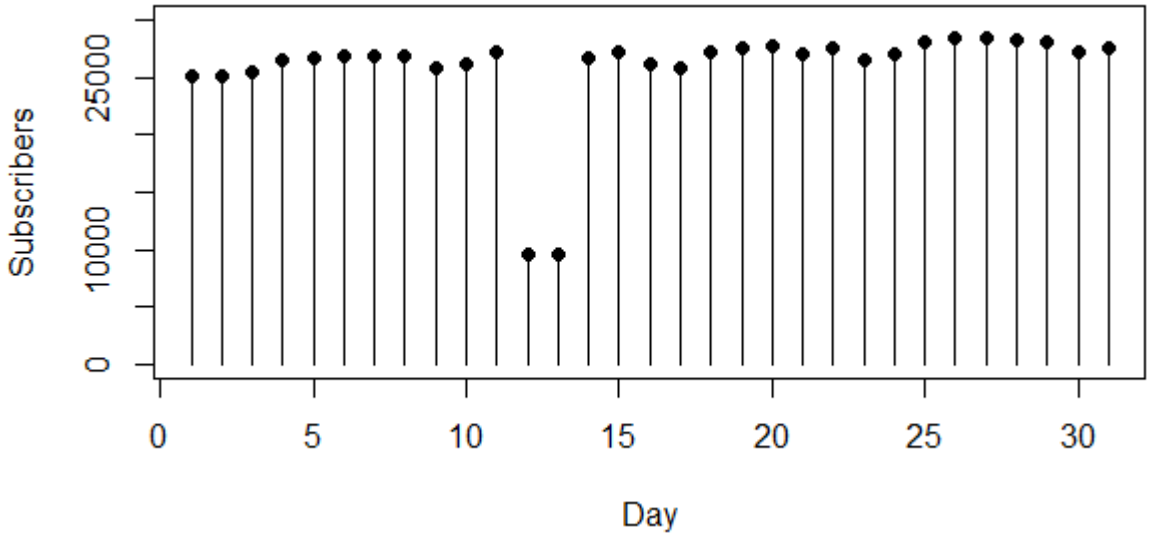


Рисунок 2.8. Комбинированная диаграмма из точек и линий

Сначала чертится диаграмма с вертикальными линиями, на этот раз с названиями осей. Затем к существующей диаграмме добавляются точки. Аргумент pch отвечает за размер, а аргумент col, который мы уже использовали со столбцовой диаграммой, определяет цвет заливки. Давайте вернемся к рис. 6. Сохраните диаграмму как PDF и откройте ее в Illustrator, чтобы заняться дизайном. результат должен быть таким, как на рисунке 11.

Используя инструмент Select (Выделение), выберите подписи и поменяйте шрифт в них на такой, какой вам нравится. Затем разгруппируйте их так, чтобы вы могли редактировать подписи по вертикальной оси по одной. Используйте Transform → Transform Each (Трансформировать → Трансформировать каждый), чтобы развернуть подписи по горизонтали. Затем возьмите инструмент Direct Selection (Частичное выделение) и удалите вертикальную линию оси — она вам не нужна, только место занимает. И наконец, выделите собственно точки, которые представляют собой простые белые кружочки, и используйте опции в

окне Color (Цвет), чтобы выбрать цвет для их обводки (Stroke) и заливки (Fill). Возможно, вам придется поменять схему цветов с Grayscale (шкалы полутонов серого) на CMYK (то есть Cyan — голубой, Magenta — пурпурный, Yellow — желтый, и black — черный цвет). Тогда выбор в окне Color (Цвет) станет богаче, рис.9

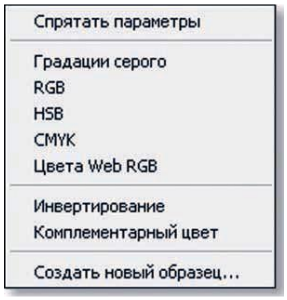


Рисунок 2.9. Меню с выбором позиций в окне Color (Цвет)

А теперь попробуйте добавить сетку таким образом, чтобы стало легче увидеть, какие значения у точек в правом углу и как они соотносятся с предыдущими точками. С помощью инструмента Select (Выделение) выделите метки на оси значений, а затем щелкните мышью и протащите их так, чтобы растянуть через всю диаграмму. В таком виде они выглядят несколько грубовато, поэтому подправьте их стиль. Как и раньше, вы можете изменить стиль линий в опциях в окне Stroke (Обводка). Используйте опции, представленные на рис. 10, если хотите сделать линии тонкими и пунктирными.

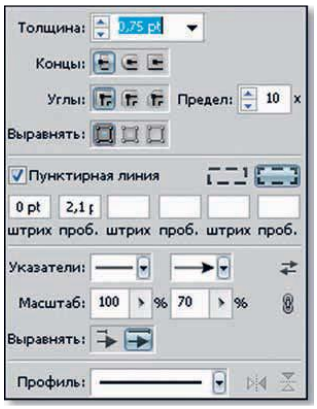


Рисунок 2.10. Опции для пунктирных линий в окне Stroke (Обводка)

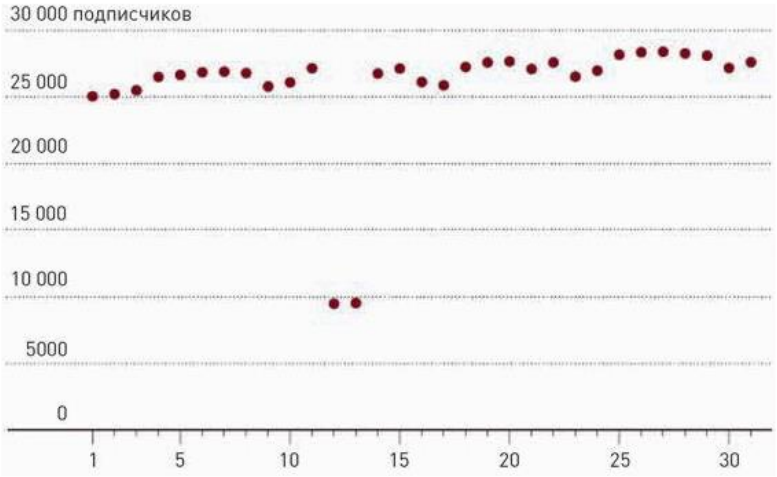


Рисунок 2.11. Конечный вариант диаграммы рассеяния.

Задание 2.2 График

Если вы знаете, как делаются диаграммы рассеяния в R, значит, вы умеете создавать и диаграммы временных рядов. Загрузите ваши данные и примените функцию plot(), но в аргументе type вместо p используйте l, что означает «линия». В качестве примера используйте данные Всемирного банка о

народонаселении мира за 1960– 2009 годы. Как обычно, загрузите данные с помощью функции read.csv(): файл world-population.csv во вложении к предыдущей лабораторной работе (данные можно получить по ссылке http://datasets.flowingdata.com/world-population.csv). После загрузки можно посмотреть первые 5 строк данных:

```
Year Population
1 1960 3028654024
2 1961 3068356747
3 1962 3121963107
4 1963 3187471383
5 1964 3253112403
```

Используйте функцию plot() и задайте x-и y-координаты, тип диаграммы, диапазон оси значений и названия осей.

```
plot(population$Year, population$Population, type="l", ylim=c(0, 7000000000), xlab="Year", ylab="Population")
```

Вы получите диаграмму, как на рисунке 12.

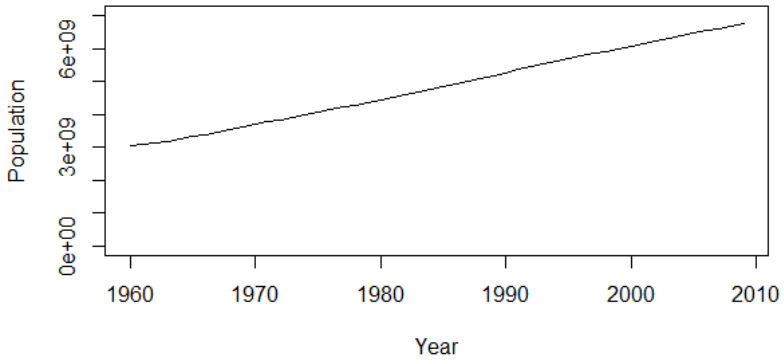


Рисунок 2.12. Диаграмма временного ряда по умолчанию в R.

На этом этапе вы уже можете сохранить изображение в формате PDF и отредактировать его в программе Illustrator, как вы делали с предыдущими двумя диаграммами. надо получить картинку похожую на рисунок 13.

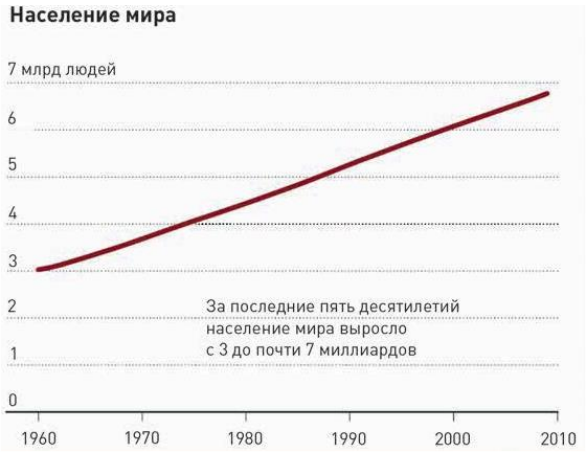


Рисунок 2.13. Изменение численности населения мира за последние пять десятилетий.

Задание2.3. Ступенчатая диаграмма.

Нарисуем ступенчатую диаграмму, как на рисунке 14. На ней показано, как менялись тарифы на пересылку писем, отправляемых через Почтовую службу США. С 1995 до 1999 года цена держалась на уровне 32 центов. То есть четыре года подряд она не менялась. А в последнее время — с 2006 по 2009 годы — изменения в тарифы вносились ежегодно.



Рисунок 2.14. Изменение почтовых ставок

Чтобы создать ступенчатую диаграмму в R, будем использовать тот же алгоритм, что и всегда (повторим!):

1. Загрузите данные.
2. Убедитесь, что данные правильно отформатированы.
3. Используйте функцию R, чтобы создать диаграмму.

Исторические данные о почтовых тарифах (наряду с множеством других сведений) вы можете найти по адресу http://datasets.flowingdata.com/us-postage.csv. Вставьте этот URL в read.csv() в качестве источника, чтобы загрузить файл с данными в R.

```
postage <- read.csv("http://datasets.flowingdata.com/us-postage.csv", sep=",", header=TRUE)
```

Ниже приводится весь набор данных. Он состоит всего из десяти пунктов, по одному на каждое изменение почтовых тарифов с 1991 по 2009 годы, а последний пункт показывает текущий уровень. В первой колонке идет год, во второй — тариф в долларах США.

```
Year Price
1  1991  0.29
2  1995  0.32
3  1999  0.33
4  2001  0.34
5  2002  0.37
6  2006  0.39
7  2007  0.41
8  2008  0.42
9  2009  0.44
10 2010  0.44
```

Функция plot() существенно облегчает создание ступенчатого графика. Как вы, наверное, и ожидали, в качестве x-координаты необходимо ввести год (Year), в качестве y-координаты — цену (Price), а аргументом type указать s, что, конечно, означает «ступенчатый».

```
plot(postage$Year, postage$Price, type="s")
```

Кроме того, вы можете ввести заголовок и названия осей — какие захотите. В качестве заголовка: «US Postage Rates for Letters, First Ounce, 1991–2010» («Почтовые тарифы на письма в США, за первые 30 граммов,

1991–2010»), а в качестве названий осей «Year» («Год») и « Postage Rate (Dollars)» («Почтовый тариф, в долларах»).

plot(postage$Year, postage$Price, type="s", main="US Postage Rates for Letters, First Ounce, 1991-2010", xlab="Year", ylab="Postage Rate (Dollars)")

В результате получится ступенчатая диаграмма почтовых тарифов, изображенную на рис.15

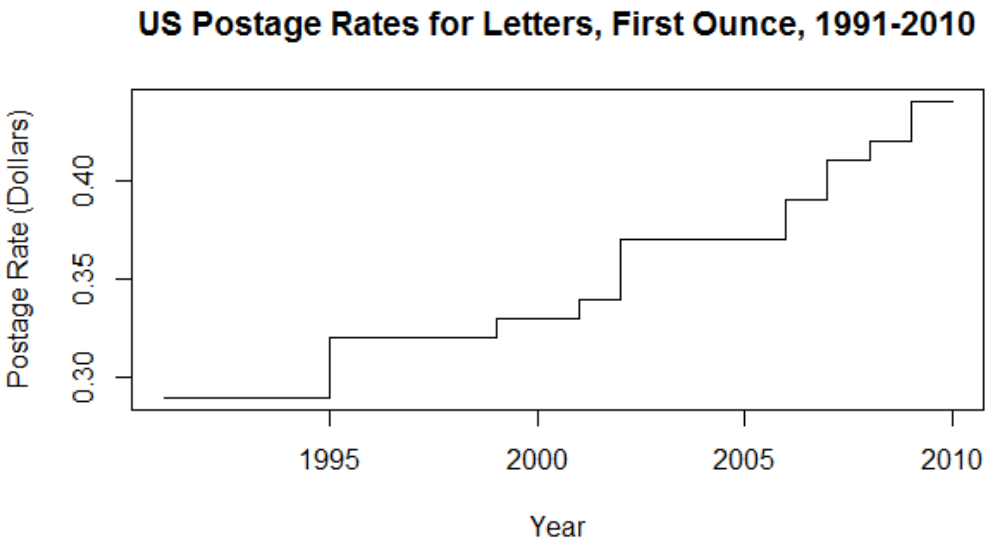


Рисунок 2.15. Ступенчатая диаграмма в R.

Попробуйте сделать этот график не ступенчатым, а линейным. На каком графике тенденция изменения тарифов видна отчетливее, по вашему мнению?

Сохраните изображение в формате PDF, а затем откройте его в программе Illustrator. Следуйте тому же алгоритму, который вы использовали ранее, и отредактируйте ступенчатую диаграмму на свой вкус.

Задание 2.4. Сглаживание и оценка в R

Порой, когда приходится работать с большими массивами данных или когда данные сопровождаются «шумом», бывает довольно сложно разглядеть в них тенденции и паттерны. Чтобы облегчить задачу, вы можете оценить линию тренда.

Попробуем нарисовать историю безработицы в США (http://datasets.flowingdata.com/unemployment-rate-1948-2010.csv) за несколько десятилетий. Отрисуйте сначала график точками, рисунок 16. Затем постройте кривую тренда (рисунок 17), для этого используйте функцию scatter.smooth().

scatter.smooth(x=1:length(unemployment$Value), y=unemployment$Value)

```
Restarting R session...

> # Load data
> unemployment <-
+     read.csv(
+         "http://datasets.flowingdata.com/unemployment-rate-1948-2010.csv",
+         sep=",")
> unemployment[1:10,]
     Series.id Year Period Value
1  LNS14000000 1948    M01   3.4
2  LNS14000000 1948    M02   3.8
3  LNS14000000 1948    M03   4.0
4  LNS14000000 1948    M04   3.9
5  LNS14000000 1948    M05   3.5
6  LNS14000000 1948    M06   3.6
7  LNS14000000 1948    M07   3.6
8  LNS14000000 1948    M08   3.9
9  LNS14000000 1948    M09   3.8
10 LNS14000000 1948    M10   3.7
> # Plain scatter plot
> plot(1:length(unemployment$Value), unemployment$Value)
>
```

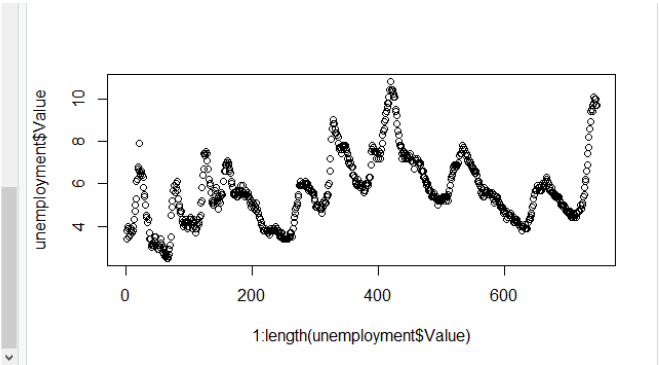


Рисунок 2.16. Диаграмма уровня безработицы с использованием точек

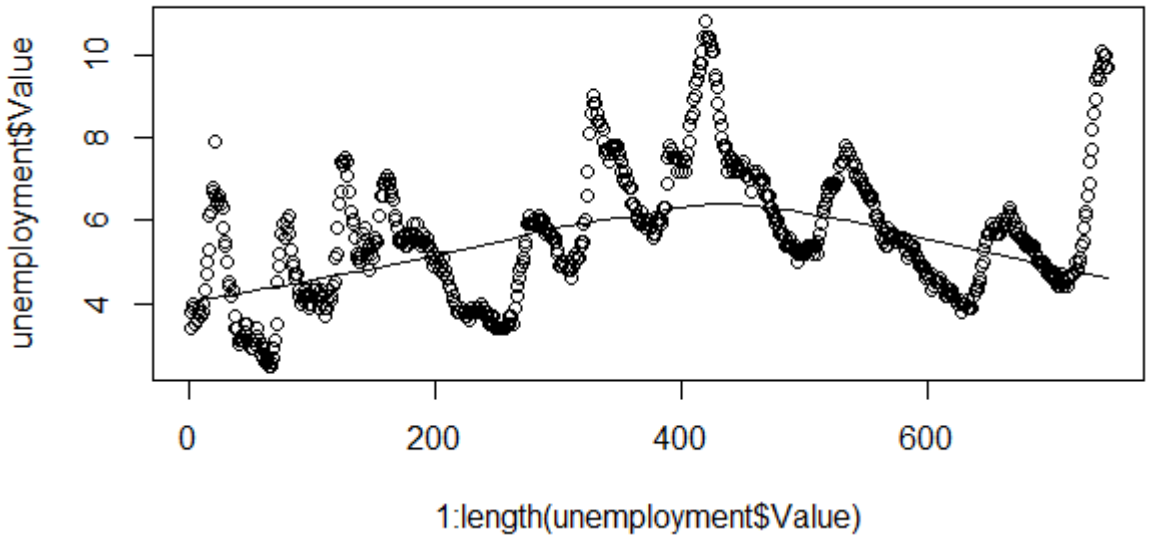


Рисунок 2.17. Кривая тренда

То, насколько сглаженной должна получиться кривая, можно задать через аргументы degree (степень) и span (интервал) в функции scatter.smooth(). Первый из них контролирует степень полиномов, а второй — то, насколько гладкой получится кривая. Чем ближе интервал к нулю, тем точнее будет подгонка. На рис. 18 показано, что именно вы получите, если поменяете степень на 2, а интервал — на 0,5. В конце измени' те цвета и настройте интервал значений осей.

scatter.smooth(x=1:length(unemployment$Value), y=unemployment$Value, ylim=c(0,11), degree=2, col="#CCCCCC", span=0.5)

С такими настройками колебания уровня безработицы на полученной кривой уже более заметны. Попробуйте поиграть с интервалом, чтобы прочувствовать, как он меняет степень сглаженности кривой. Чтобы получить конечный вариант диаграммы, представленный на рис. 19, сохраните изображение как PDF и откройте его в Illustrator. Используйте те же самые инструменты — Selection (Выделение), Type (Текст), Pen (Перо), — чтобы оформить заголовок, фон и метки на горизонтальной оси. Сглаженная линия сделана более яркой, чтобы привлекать внимание к тенденции, а не к отдельным точкам данных.



Рисунок 2.18. Сглаженная кривая с более низкой степенью сглаженности и более высокой степенью полинома



Рисунок 2.19. История безработицы в США

Изучать паттерны во времени очень интересно. Время является неотъемлемой частью нашей повседневной жизни и так тесно переплетено с ней. Вы знаете, что все меняется и развивается, и сложность здесь состоит в другом — определить степень этих изменений и научиться понимать, что именно следует искать в своих диаграммах. Это нетрудно — бросить взгляд на диаграмму и сказать, что в чем-то наблюдается рост и что это хорошо. Собственно, для того и существует визуализация: она позволяет быстро составить общее впечатление о картине, складывающейся из массива данных. Но вы можете пойти дальше. Вы можете использовать визуализацию в качестве исследовательского инструмента. Рассмотреть в приближении те или иные отрезки времени и задаться вопросом: почему в такой-то день произошел маленький всплеск, какого больше ни разу не наблюдалось, или почему в другой день произошел такой-то перепад. Вот тогда данные станут интересными и увлекательными — а чем больше вы знаете о своих данных, тем лучше получится у вас история, которую собираетесь рассказать. Когда вы разберетесь, о чем на самом деле говорят ваши данные, тогда и поясните эти детали в инфографике. Выделите интересные элементы, чтобы ваши читатели поняли, на что смотреть. Может, диаграмма без подписей говорит вам о многом, но для остальных без контекста она будет безликой. Чтобы добиться достойных результатов, вы использовали R и Illustrator. В R вы создавали основу, а Illustrator применяли для оформления диаграмм, чтобы показать, что именно важно в собранных данных. Рассмотренные нами типы диаграмм представляют собой, конечно, только небольшое подмножество того, что вы можете делать с темпоральными данными. Вы откроете еще целый набор хитроумных приемов, когда присовокупите к уже освоенным методам анимацию и интерактивность.

Часть 2. Визуализация зависимостей

Задание 1. Диаграмма рассеяния

Рассмотрим уровень преступности в США по штатам за 2005 год в пересчете на 100 000 человек (по данным Бюро переписи населения). Всего мы рассмотрим семь типов преступлений, а именно: убийства (murder), изнасилования (forcible rape), грабежи (robbery), нападения с применением физического насилия (aggravated assault), квартирные кражи со взломом (burglary), кражи имущества (larceny-theft) и кражи транспортных средств (motor vehicle theft).

1. Первым делом необходимо загрузить данные в R с помощью read.csv(). Нужный CSV-файл вы можете найти по адресу http://datasets.flowingdata.com/crimeRatesByState2005.csv. Это можно сделать так:

```
# Load the data
crime <-
   read.csv('http://datasets.flowingdata.com/crimeRatesByState2005.csv',
   sep=",", header=TRUE)
```

2. Проверьте первые несколько строчек данных, набрав переменную **crime** и следом за ней указав строчки, которые хотите посмотреть.

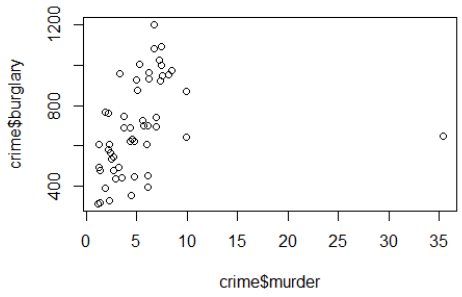
```
> crime [1:5,]
```

Отобразится следующая таблица:

```
> # Load the data
> crime <-
+     read.csv('http://datasets.flowingdata.com/crimeRatesByState2005.csv',
+              sep=",", header=TRUE)
> crime [1:5,]
          state murder forcible_rape robbery aggravated_assault burglary larceny_theft
1 United States    5.6          31.7   140.7               291.1    726.7        2286.3
2       Alabama    8.2          34.3   141.4               247.8    953.8        2650.0
3        Alaska    4.8          81.1    80.9               465.1    622.5        2599.1
4       Arizona    7.5          33.8   144.4               327.4    948.4        2965.2
5      Arkansas    6.7          42.9    91.1               386.8   1084.6        2711.2
  motor_vehicle_theft population
1               416.7  295753151
2               288.3    4545049
3               391.0     669488
4               924.4    5974834
5               262.1    2776221
>
```

3. В первой колонке идет название штата, а далее представлены данные по различным типам преступлений. Например, средний по США уровень преступности по части грабежей в 2005 году составлял 140,7 на 100 000 жителей. Используя plot(), создайте диаграмму рассеяния по умолчанию, сопоставив данные по убийствам (murder) и по кражам со взломом (burglary):

```
> plot(crime$murder, crime$burglary)
```



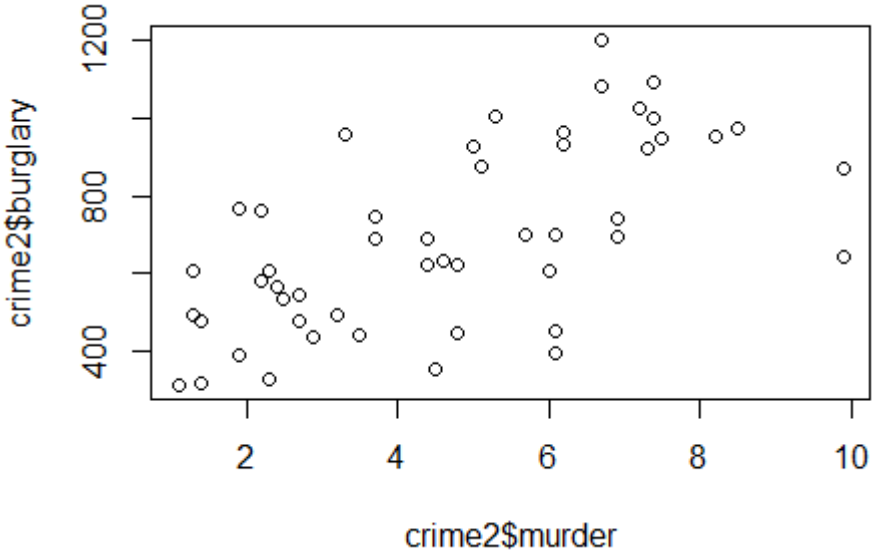
Как называется данный вид зависимости?
Какой тип корреляции получился?

Штаты, в которых уровень преступности в части убийств выше, склонны демонстрировать также и более высокий уровень по кражам со взломом, но разглядеть данную тенденцию непросто из-за одной точки на правой стороне диаграммы. Эта одинокая точка — или выброс — вынуждает горизонтальную ось сильно вытягиваться в ширину. Точка представляет Вашингтон, округ Колумбия , где уровень по убийствам весьма высок — 35,4. В следующих сразу за ним штатах — Луизиане и Мэриленде — данный показатель составляет 9,9. Чтобы диаграмма получилась более наглядной и полезной, удалите Вашингтон, а заодно, пока вы все еще там, удалите и средние по США показания и сконцентрируйтесь только на данных по отдельным штатам. Чтобы диаграмма получилась более наглядной и полезной, удалите Вашингтон, а

заодно, пока вы все еще там, удалите и средние по США показания и сконцентрируйтесь только на данных по отдельным штатам:

```
crime2 <- crime[crime$state != "District of Columbia",]
crime2 <- crime2[crime2$state != "United States",]
```

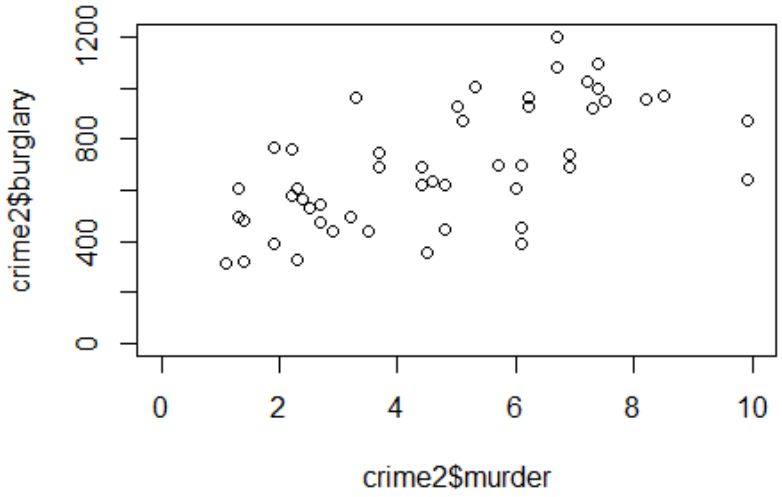
Первая строчка сохраняет все ряды, которые не касаются округа Колумбия в crime2. Аналогичным образом вторая строчка отфильтровывает средние значения по Соединенным Штатам в целом. Теперь, выстроив диаграмму, сопоставляющую убийства с кражами со взломом, вы получите уже более ясную картину, как показано на рисунке ниже.



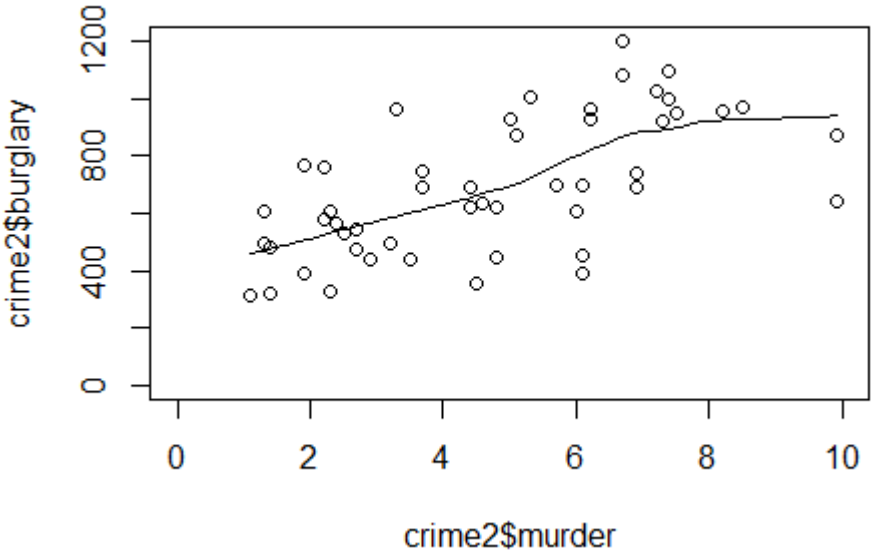
Хотя было бы лучше, если б оси начинались с нуля, так что подкорректируйте и это тоже. Ось X должна иметь охват от 0 до 10, а ось Y — от 0 до 1200:

```
plot(crime2$murder, crime2$burglary, xlim=c(0,10), ylim=c(0,1200))
```

Таким образом точки сместятся выше и правее, как на рисунке:



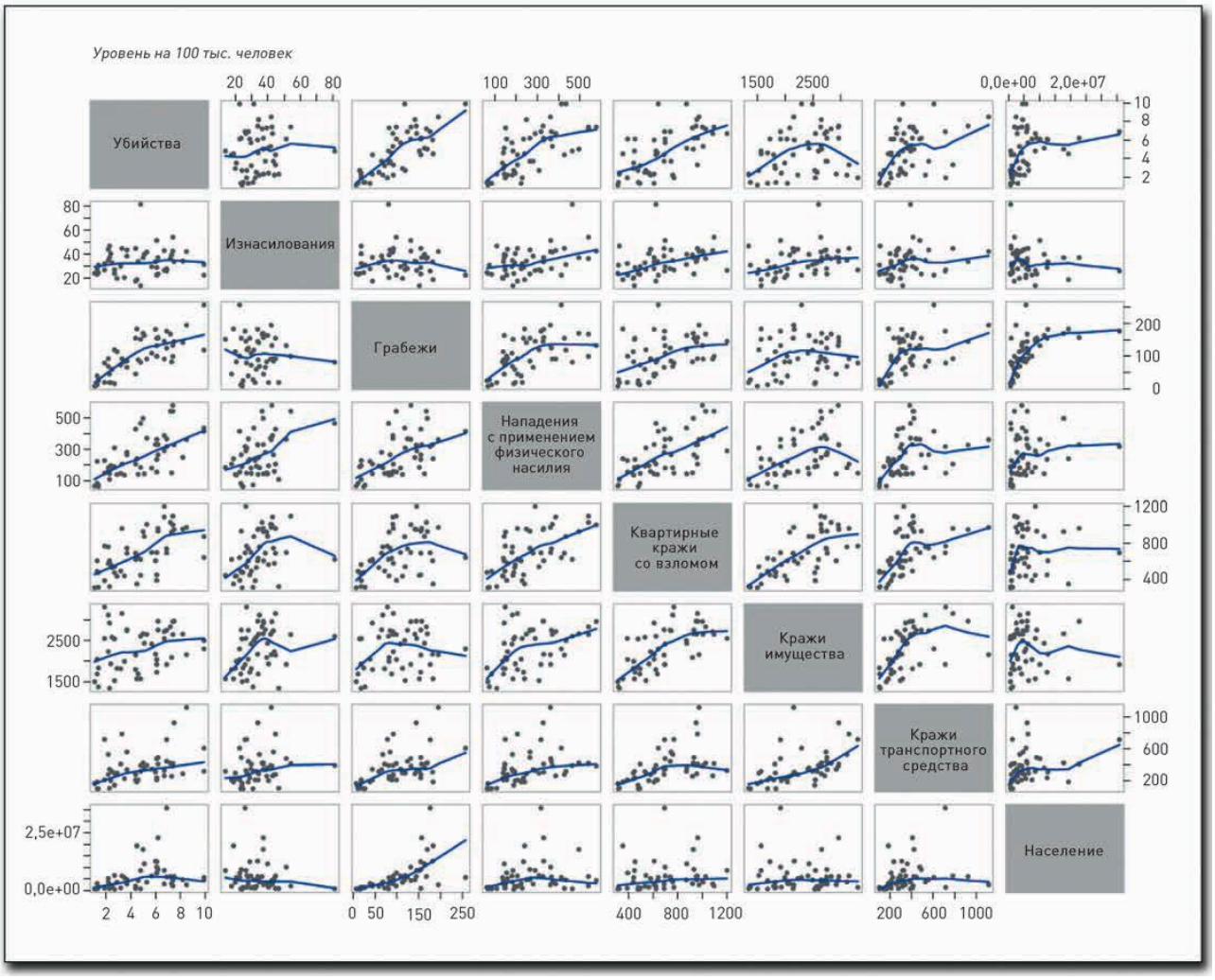
Сделайте эту диаграмму еще более наглядной с помощью кривой, проведенной между точками, используя scatter.smooth:



Задание 2. Матрица диаграмм рассеяния

Каждая колонка в матрице рассеяния представляет одну переменную по горизонтальной оси, а каждый ряд — одну переменную по вертикальной оси. Таким образом вы получаете все возможные пары, а диагональ остается для подписей, так как нет никакого смысла сравнивать переменную с самой собой.

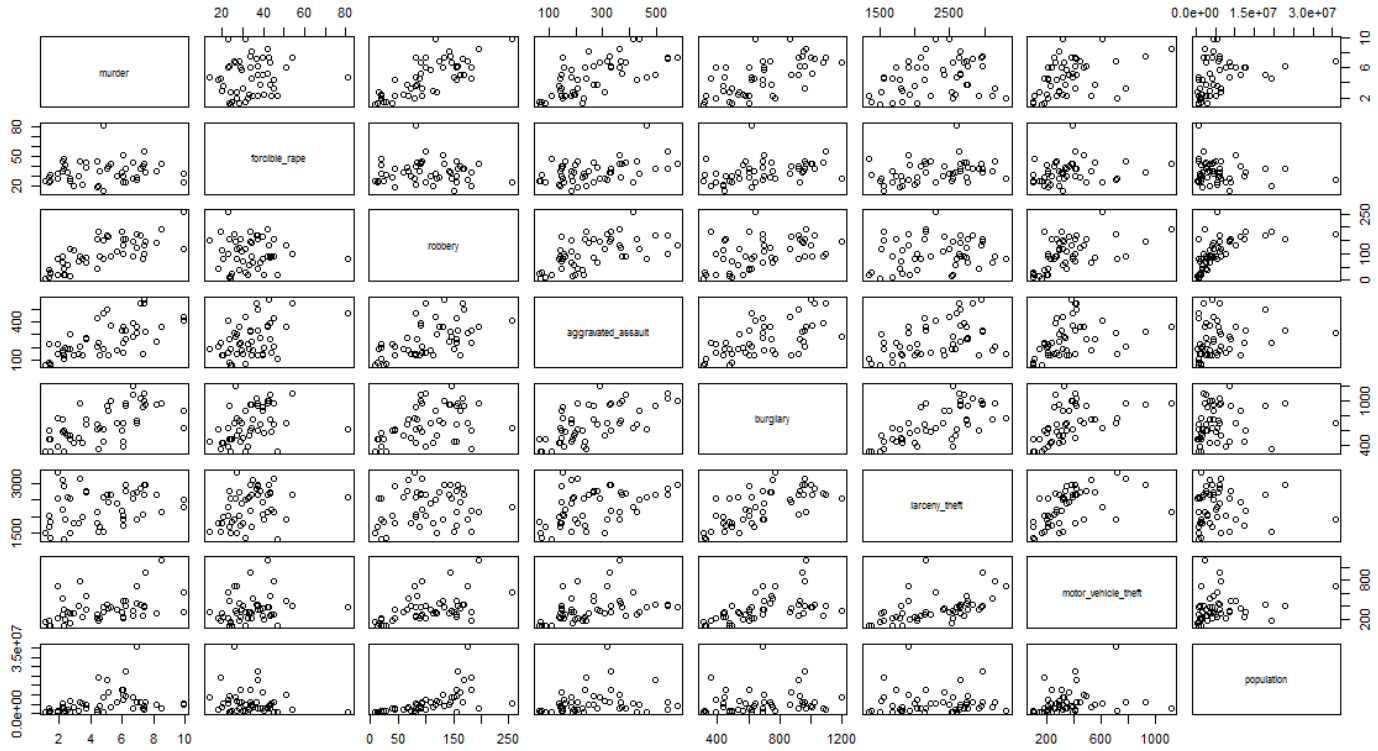
Теперь давайте вернемся к данным о преступности. У вас есть семь переменных (или показателей по разным типам преступлений), но в предыдущем примере вы сравнивали только два из них: убийства и кражи со взломом. Теперь же с матрицей диаграмм рассеяния вы можете сопоставить все виды преступлений.



Налицо множество положительных корреляций. Например, между кражами со взломом и нападениями с применением физического насилия. А вот связь между убийствами и кражами имущества не так очевидна. Такую диаграмму надо читать слева направо и сверху вниз.

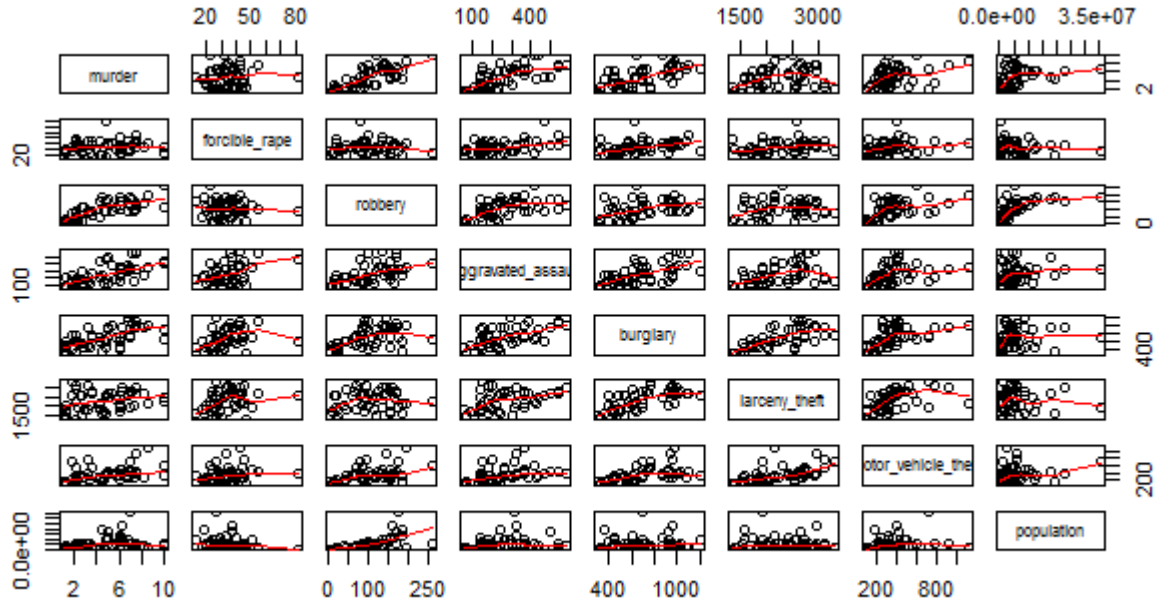
R превращает создание матричных диаграмм рассеяния в столь же простое дело, как и начертание единичной диаграммы, хотя процесс создания матрицы протекает все же не так быстро. Вы снова воспользуетесь функцией plot(), но вместо двух колонок загрузите всю таблицу данных за исключением первой колонки, так как она содержит только названия штатов:

```
plot(crime2[,2:9])
```



Чтобы создать матрицу со сглаженными кривыми, вместо функции, к которой вы обращались перед этим, на сей раз вы воспользуетесь функцией pairs():

```
pairs(crime2[,2:9], panel=panel.smooth)
```



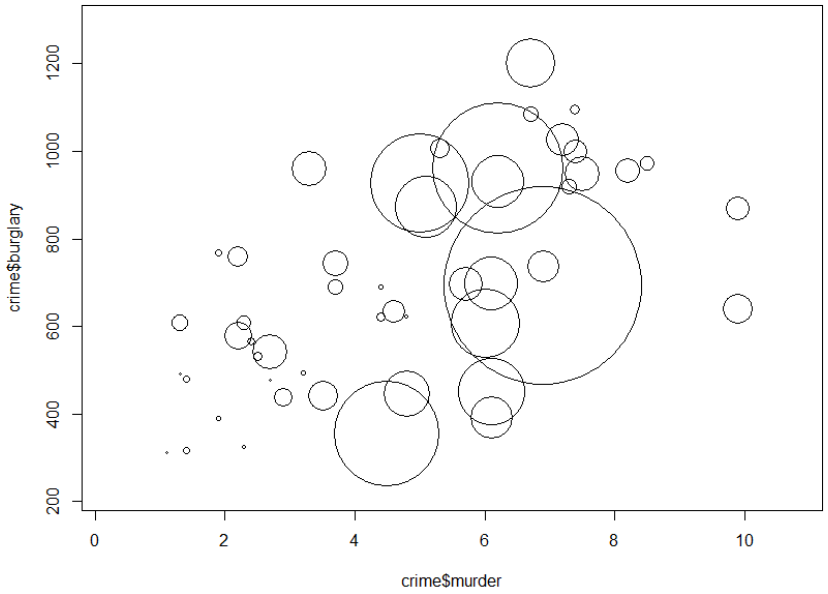
Задание 3. Создайте пузырьковую диаграмму

Загрузите данные в R с помощью read.csv(). Это те же самые данные, которые вы только что использовали, с той лишь разницей, что здесь также добавлена колонка с численностью населения и в массив не включен Вашингтон. А еще в данном случае в качестве разделителя в файле используется табуляция, а не запятые. Ничего страшного. Просто измените в функции аргумент sep:

```
crime <-
    read.csv("http://datasets.flowingdata.com/crimeRatesByState2005.tsv",
             header=TRUE, sep="\t")
```

А затем сразу же приступайте к созданию пузырьков с помощью функции symbols(). На оси X представлен уровень убийств, на оси Y — уровень краж со взломом, а радиус пузырьков, соответственно, пропорционален численности населения в различных штатах:

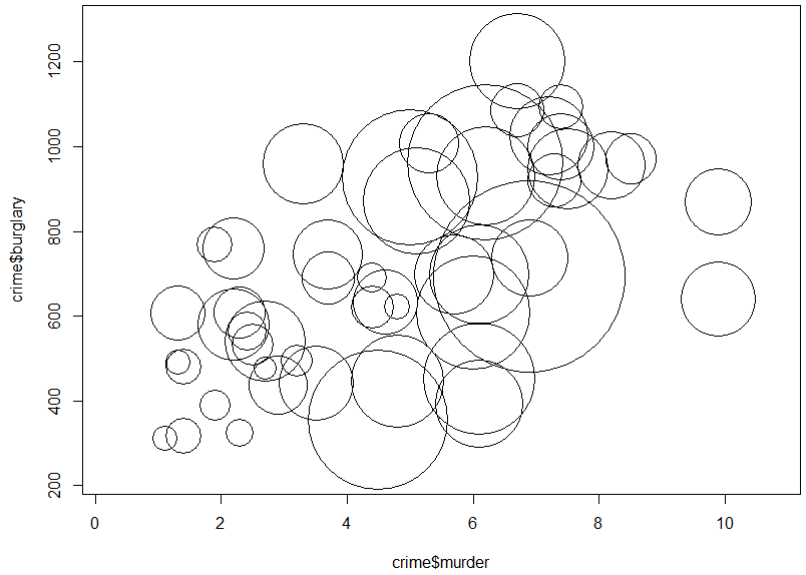
```
symbols(crime$murder, crime$burglary, circles=crime$population)
```



Думаете, все получилось правильно? Сейчас размер кругов определяется таким образом, что их радиус прямо пропорционален численности населения. А как должно быть? Нам необходимо, чтобы площадь, а не радиус, была пропорциональна численности населения. Чтобы определить размер радиусов корректно, вспомните уравнение для вычисления площади круга. Площадь каждого пузырька выражает численность населения. А вам нужно выяснить, как определить размер радиуса. Так вот: выделите радиус, и вы поймете, что он должен быть пропорционален квадратному корню из площади круга. А теперь, чтобы определить радиусы кругов, используйте не crime$population, а извлеките квадратный корень и затем вставьте его в symbols():

```
radius <- sqrt( crime$population/ pi )
symbols(crime$murder, crime$burglary, circles=radius)
```

Получим такую картинку:



Чтобы увидеть ясную картину и понять, что происходит, вам необходимо уменьшить размер всех кругов. Аргумент inches в symbols() задает размер самого большого круга в дюймах. По умолчанию он устанавливается на уровне 1 дюйм (2,54 см), а потому на рисунке, обозначающий Калифорнию круг, получил размер в 1 дюйм, а все остальные были подстроены под него. Но вы можете сделать максимальный размер меньше — скажем, 0,35 дюйма (0,89 см), и при этом сохранить правильные пропорции. Вы можете также переопределить цвет, используя fg и bg, чтобы задать цвета обводки и заливки соответственно. А еще вы можете добавить к осям свои собственные подписи:

```
.3
2       Alabama    8.2          34.3   141.4              247.8    953.8        2650
.0
3        Alaska    4.8          81.1    80.9              465.1    622.5        2599
.1
4       Arizona    7.5          33.8   144.4              327.4    948.4        2965
.2
5      Arkansas    6.7          42.9    91.1              386.8   1084.6        2711
.2
  motor_vehicle_theft population
1               416.7  295753151
2               288.3    4545049
3               391.0     669488
4               924.4    5974834
5               262.1    2776221
> plot(crime$murder, crime$burglary)
> crime2 <- crime[crime$state != "District of Columbia",]
> crime2 <- crime2[crime2$state != "United States",]
> plot(crime$murder, crime$burglary)
> plot(crime2$murder, crime2$burglary)
> plot(crime2$murder, crime2$burglary, xlim=c(0,10), ylim=c(0,1200))
> scatter.smooth(crime2$murder, crime2$burglary,
+                xlim=c(0,10), ylim=c(0,1200))
> plot(crime2[,2:9])
> pairs(crime2[,2:9], panel=panel.smooth)
>
> symbols(crime$murder, crime$burglary, circles=crime$population)

Restarting R session...

> crime <-
+     read.csv("http://datasets.flowingdata.com/crimeRatesByState2005.tsv",
+              header=TRUE, sep="\t")
> symbols(crime$murder, crime$burglary, circles=crime$population)
> radius <- sqrt( crime$population/ pi )
> symbols(crime$murder, crime$burglary, circles=radius)
> symbols(crime$murder, crime$burglary, circles=radius, inches=0.35,
+         fg="white", bg="red", xlab="Murder Rate", ylab="Burglary Rate")
>
```

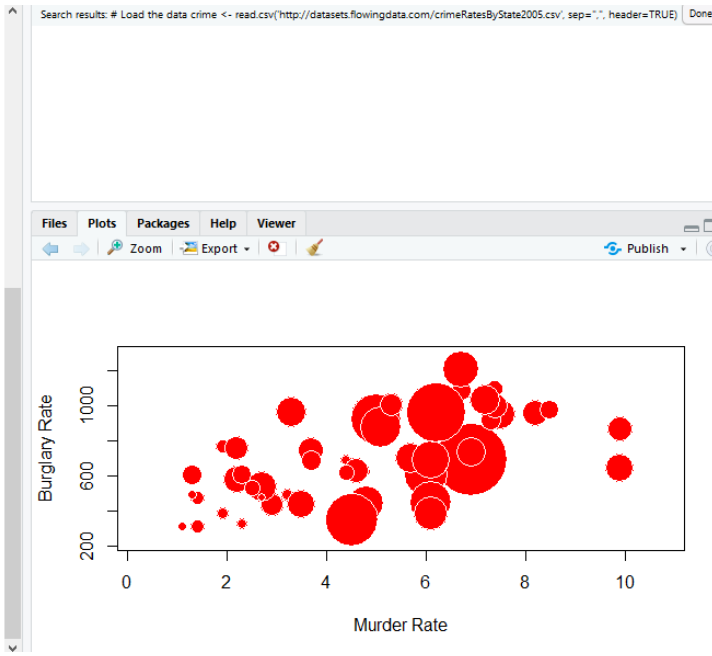
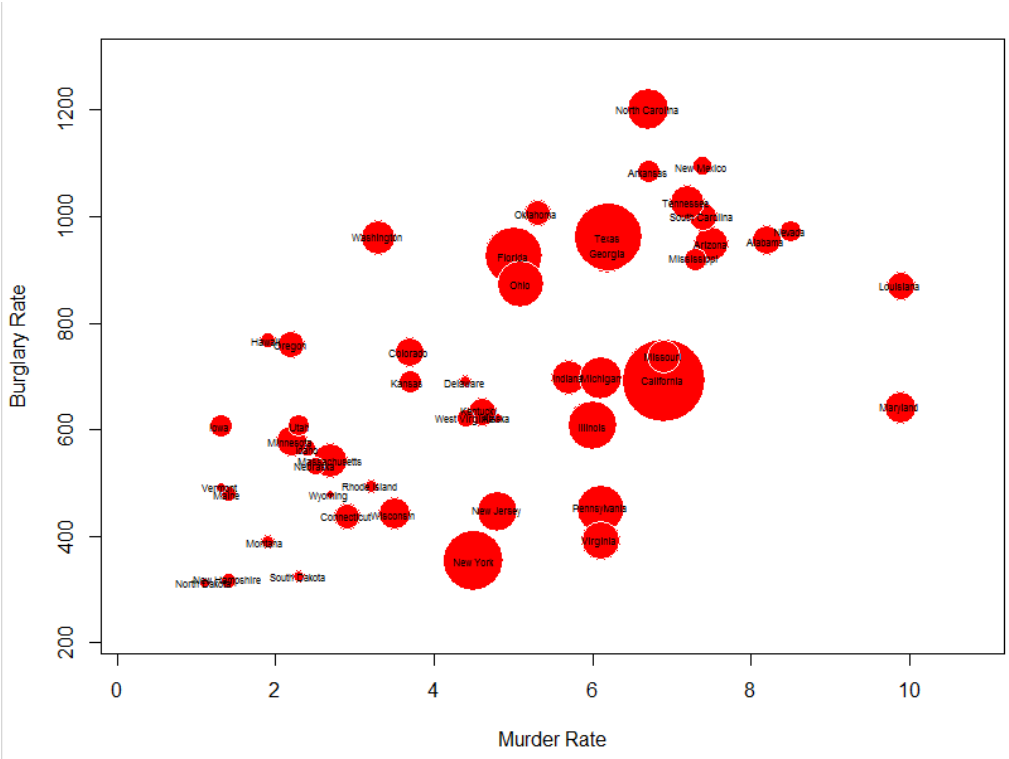


Диаграмма на рисунке выше дает некое представление о распределении, но вы не знаете, какой круг представляет тот или иной штат. А потому следует добавить подписи с помощью text(), где аргументами будут выступать x-координаты, y-координаты и собственно текст. Все это у вас есть. Как и с пузырьками, по оси X пойдут убийства, а по Y — кражи со взломом. В качестве подписей будут выступать названия штатов, которые приведены в первой колонке таблицы с данными:

text(crime$murder, crime$burglary, crime$state, cex=0.5)

Аргумент cex контролирует размер текста, по умолчанию он равен 1. Значения больше единицы делают подписи крупнее, и наоборот — меньшие значения делают шрифт мельче. Подписи можно центрировать по x- и y-координатам.

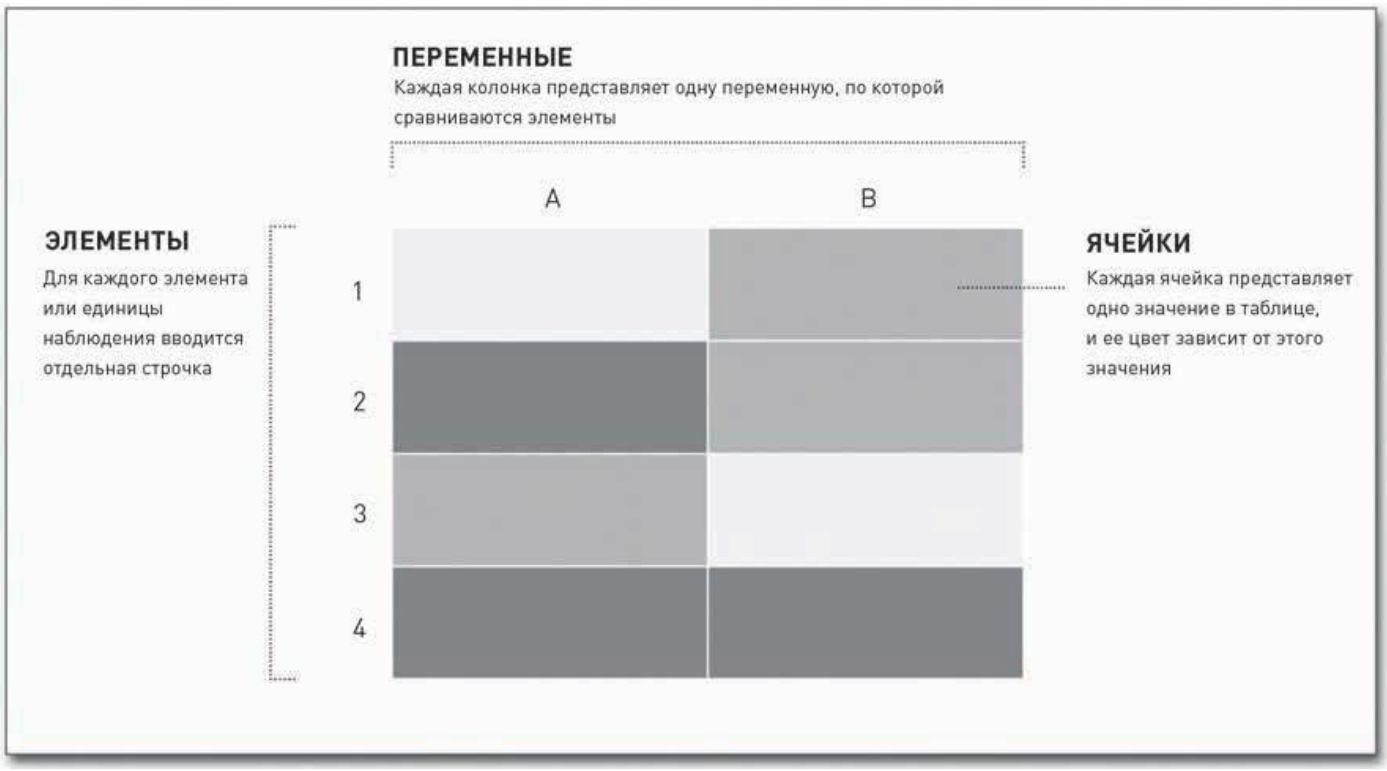


А теперь наберите ?symbols в R, чтобы посмотреть и другие опции для создания диаграмм. Нарисуйте вместо кругов диаграмму из квадратов.

Часть 3. Выявление различий

Задание 1. Создание тепловой карты

Тепловую карту (тепловую матрицу) читают так же, как и таблицу. Вы можете прочитать строчки слева направо, чтобы увидеть значения всех переменных для одного отдельного элемента, или можете просмотреть все элементы в сравнении друг с другом по одной переменной.



Такой способ представления данных, скорее всего, покажется вам несколько сложным и запутанным, особенно если таблица данных большая, но, подобрав правильную цветовую схему и немного поработав над сортировкой данных, вы можете создать графику, которая сослужит вам хорошую службу.

Создавать тепловые карты в R проще простого. Для этого существует функция heatmap(), которая выполнит все математические операции, а вам останется лишь подобрать подходящие для ваших данных цвета и организовать подписи таким образом, чтобы они оставались читаемыми, даже если строк и колонок будет много. Иными словами, R определит структуру, а вы займетесь дизайном.

В данном примере мы обратимся к статистике игр Национальной баскетбольной ассоциации за 2008 год. Данные вы можете скачать в виде CSV-файла с http://datasets.flowingdata.com/ppg2008.csv . У вас 22 колонки: в первой даны имена игроков, а далее идет статистика — очки за игру, процент трехочковых бросков и т. д. Чтобы загрузить данные в R, можете воспользоваться read.csv(). Затем, как мы делали это ранее, посмотрите на первые пять строчек, чтобы создать себе представление о структуре данных, см. рисунок ниже.

```
> bball<- read.csv("http://datasets.flowingdata.com/ppg2008.csv", sep=",", header=TR
UE)
> bball[1:5,]
            Name  G  MIN  PTS  FGM  FGA   FGP FTM FTA   FTP X3PM X3PA  X3PP ORB
1   Dwyane Wade  79 38.6 30.2 10.8 22.0 0.491 7.5 9.8 0.765  1.1  3.5 0.317 1.1
2  LeBron James  81 37.7 28.4  9.7 19.9 0.489 7.3 9.4 0.780  1.6  4.7 0.344 1.3
3   Kobe Bryant  82 36.2 26.8  9.8 20.9 0.467 5.9 6.9 0.856  1.4  4.1 0.351 1.1
4 Dirk Nowitzki  81 37.7 25.9  9.6 20.0 0.479 6.0 6.7 0.890  0.8  2.1 0.359 1.1
5 Danny Granger  67 36.2 25.8  8.5 19.1 0.447 6.0 6.9 0.878  2.7  6.7 0.404 0.7
  DRB TRB AST STL BLK  TO  PF
1 3.9 5.0 7.5 2.2 1.3 3.4 2.3
2 6.3 7.6 7.2 1.7 1.1 3.0 1.7
3 4.1 5.2 4.9 1.5 0.5 2.6 2.3
4 7.3 8.4 2.4 0.8 0.8 1.9 2.2
5 4.4 5.1 2.7 1.0 1.4 2.5 3.1
```

В настоящий момент игроки выстроены по очкам за игру от наибольшего значения к наименьшему, но вы можете рассортировать их по показателям любой колонки, например по количеству подборов мяча при отскоке или по проценту попаданий с игры, с помощью order(). Посмотрите на первые пять строк bball_byfgp, вы увидите, что список возглавляют Шакил О'Нил, Дуайт Ховард и По Газоль, а не Дуэйн Уэйд, ЛеБрон Джеймс и Коуб Брайант.

```
> bball_byfgp <- bball[order(bball$FGP, decreasing=TRUE),]
> bball_byfgp[1:5,]
                Name  G  MIN  PTS FGM  FGA   FGP FTM  FTA   FTP X3PM X3PA  X3PP
46 Shaquille O'neal  75 30.1 17.8 6.8 11.2 0.609 4.1  6.9 0.595  0.0  0.0 0.000
25    Dwight Howard  79 35.7 20.6 7.1 12.4 0.572 6.4 10.7 0.594  0.0  0.0 0.000
36        Pau Gasol  81 37.1 18.9 7.3 12.9 0.567 4.2  5.4 0.781  0.0  0.0 0.500
29         Yao Ming  77 33.6 19.7 7.4 13.4 0.548 4.9  5.7 0.866  0.0  0.0 1.000
15 Amare Stoudemire  53 36.8 21.4 7.6 14.1 0.539 6.1  7.3 0.835  0.1  0.1 0.429
   ORB DRB  TRB AST STL BLK  TO  PF
46 2.5 5.9  8.4 1.7 0.7 1.4 2.2 3.4
25 4.3 9.6 13.8 1.4 1.0 2.9 3.0 3.4
36 3.2 6.4  9.6 3.5 0.6 1.0 1.9 2.1
29 2.6 7.2  9.9 1.8 0.4 1.9 3.0 3.3
15 2.2 5.9  8.1 2.0 0.9 1.1 2.8 3.1
```

Но для данного примера вам необходимо выстроить игроков обратно по количеству очков за игру.

```
> bball <- bball[order(bball$PTS, decreasing=FALSE),]
>  bball[1:5,]
                Name  G  MIN  PTS FGM  FGA   FGP FTM FTA   FTP X3PM X3PA  X3PP
50    Nate Robinson  74 29.9 17.2 6.1 13.9 0.437 3.4 4.0 0.841  1.7  5.2 0.325
49    Allen Iverson  57 36.7 17.5 6.1 14.6 0.417 4.8 6.1 0.781  0.5  1.7 0.283
47    Rashard Lewis  79 36.2 17.7 6.1 13.8 0.439 2.8 3.4 0.836  2.8  7.0 0.397
48 Chauncey Billups  79 35.3 17.7 5.2 12.4 0.418 5.3 5.8 0.913  2.1  5.0 0.408
45 Maurice Williams  81 35.0 17.8 6.5 13.9 0.467 2.6 2.8 0.912  2.3  5.2 0.436
   ORB DRB TRB AST STL BLK  TO  PF
50 1.3 2.6 3.9 4.1 1.3 0.1 1.9 2.8
49 0.5 2.5 3.0 5.0 1.5 0.1 2.6 1.5
47 1.2 4.6 5.7 2.6 1.0 0.6 2.0 2.5
48 0.4 2.6 3.0 6.4 1.2 0.2 2.2 2.0
45 0.6 2.9 3.4 4.1 0.9 0.1 2.2 2.7
>
```

Еще необходимо вместо номеров присвоить строчкам имена игроков, а потому перенесите первую колонку в названия строк:

```
row.names(bball) <- bball$Name
bball <- bball[,2:20]
```

В данном фрагменте сначала названия строк меняются на первую колонку таблицы данных, затем выбираются все колонки со 2-й по 20-ю, и это подмножество данных пересылается обратно в bball.

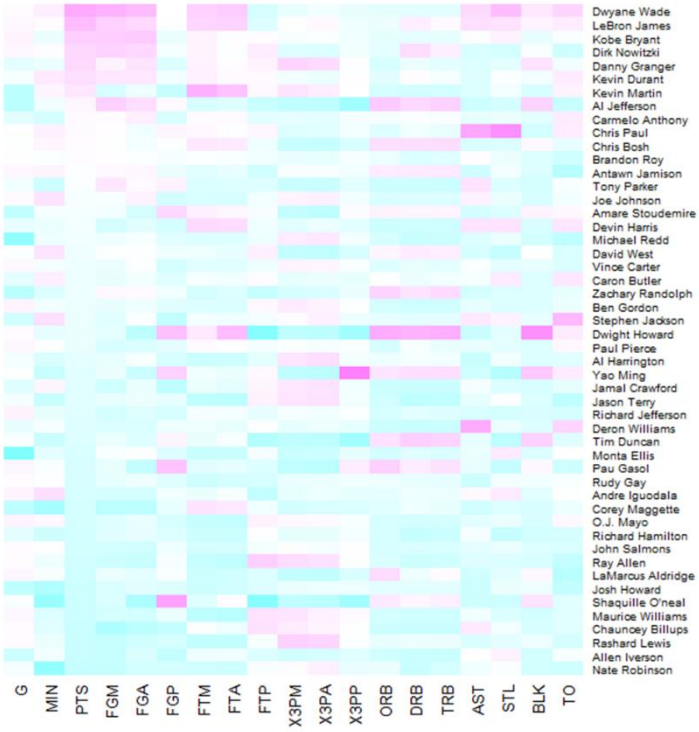
Кроме того, надо, чтобы данные были скорее в формате матрицы, нежели просто таблицы. Если вы попробуете использовать таблицу данных с функцией heatmap(), то вы получите ошибку. По сути, таблица данных — это набор векторов, в котором каждая колонка представляет те или иные количественные показатели. У колонок могут быть различные форматы — числа или некая другая последовательность символов. Матрица же используется, как правило, для представления двухмерного пространства, и данные в ней, от первой до последней ячейки, должны быть единообразны по своему типу.

```
bball_matrix <- data.matrix(bball)
```

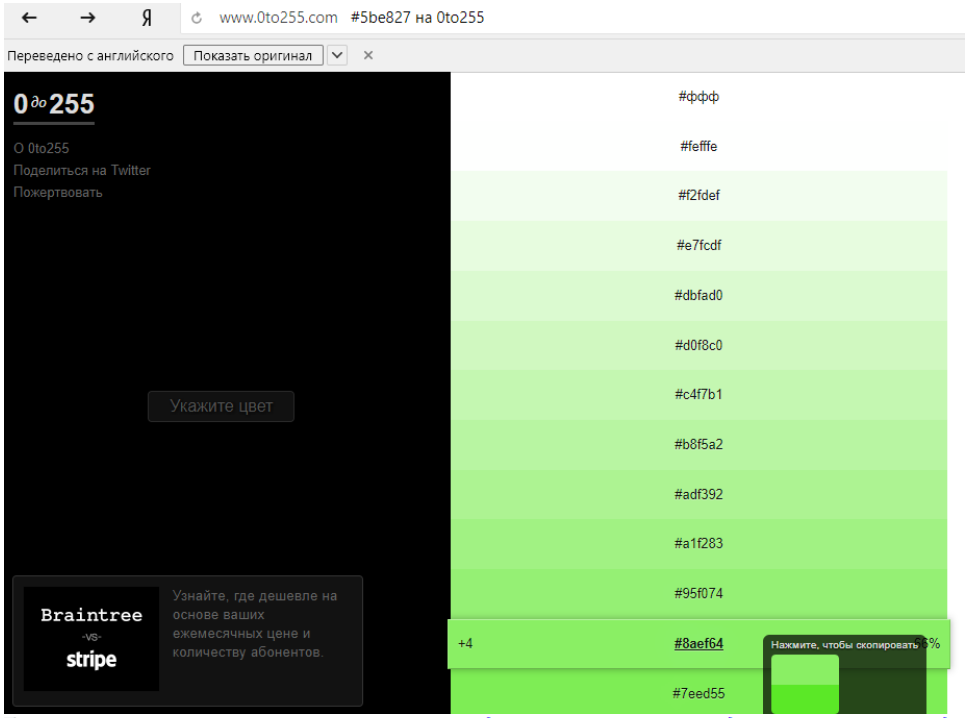
Теперь данные выстроены в нужном нам порядке. Можно передать их в heatmap().

```
ball_heatmap <- heatmap(bball_matrix, Rowv=NA, Colv=NA, col = cm.colors(256), scale="column", margins=c(5,10))
```

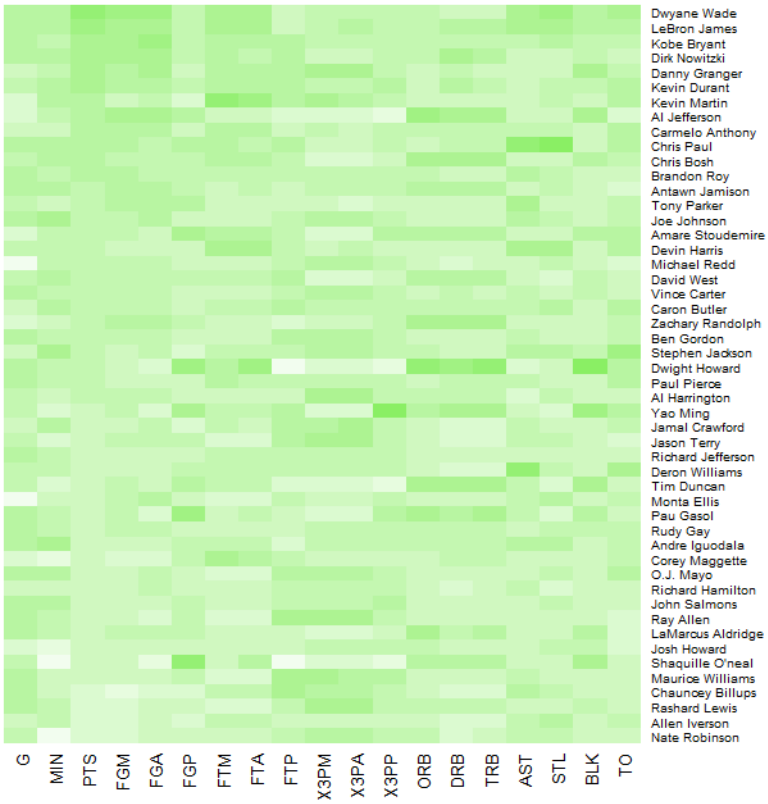
Когда для аргумента scale вы задаете значение “column”, вы тем самым поручаете R при определении цветового градиента использовать наибольшие и наименьшие значения каждой колонки, а не минимум и максимум всей матрицы. С помощью cm.colors() был установлен цветовой диапазон от голубого (cyan) до пурпурного (magenta). Функция создает по умолчанию вектор шестнадцатеричных цветов в диапазоне от голубого до пурпурного, при этом количество оттенков между ними можно менять (в данном случае их 256). Обратите внимание на то, что третья колонка (в которой представлено количество очков за игру) начинается с пурпурного, означающего самые высокие показатели Дуэйна Уэйда и Леброна Джеймса, и затем медленно переходит в голубые оттенки до самого темного цвета в конце колонки, где представлены показатели Аллена Айверсона и Нэйта Робинсона. Так же легко вы можете заметить и другие ярко-пурпурные ячейки, представляющие лучшие показатели по отскокам (у Дуайта Ховарда) и лучшего игрока по передачам (это Крис Пол).



Вы можете сходить на сайт 0to255.com и там выбрать главный цвет (я сделала базовым зеленый), а затем на его основе выстроить все остальное. Вместо того чтобы использовать R для создания вектора цветов, в переменной g_colors нужно указать свои собственные значения.



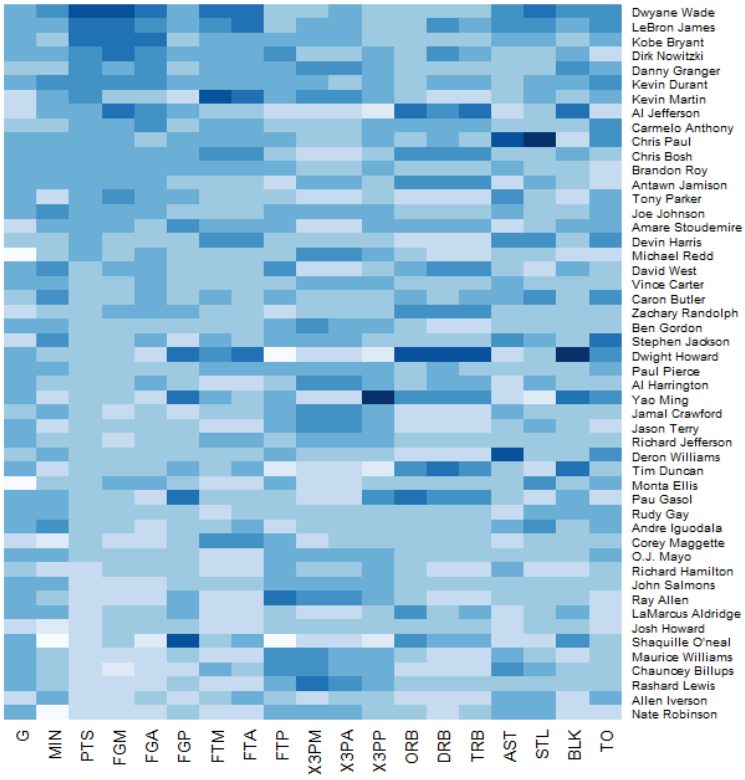

```
> g_colors <- c("#f2fdef", "#e7fcdf", "#dbfad0", "#d0f8c0", "#c4f7b1", "#b8f5a2", "#adf392", "#a1f283",
"#95f074", "#8aef64")
> bball_heatmap <- heatmap(bball_matrix, Rowv=NA,
+   Colv=NA, col = g_colors, scale="column", margins=c(5,10))
```



Если вы хотите самостоятельно подбирать цвета, можете воспользоваться пакетом RColorBrewer. Он не стоит по умолчанию, так что вам придется загрузить и установить его с помощью Package Installer. С ним вы можете выбирать из множества опций: там есть и последовательные цветовые палитры, и расходящиеся, а также огромное количество оттенков. Для целей нашего примера подойдет и простая голубая палитра. Наберите ?brewer.pal в консоли R, чтобы ознакомиться с дополнительными опциями — это довольно

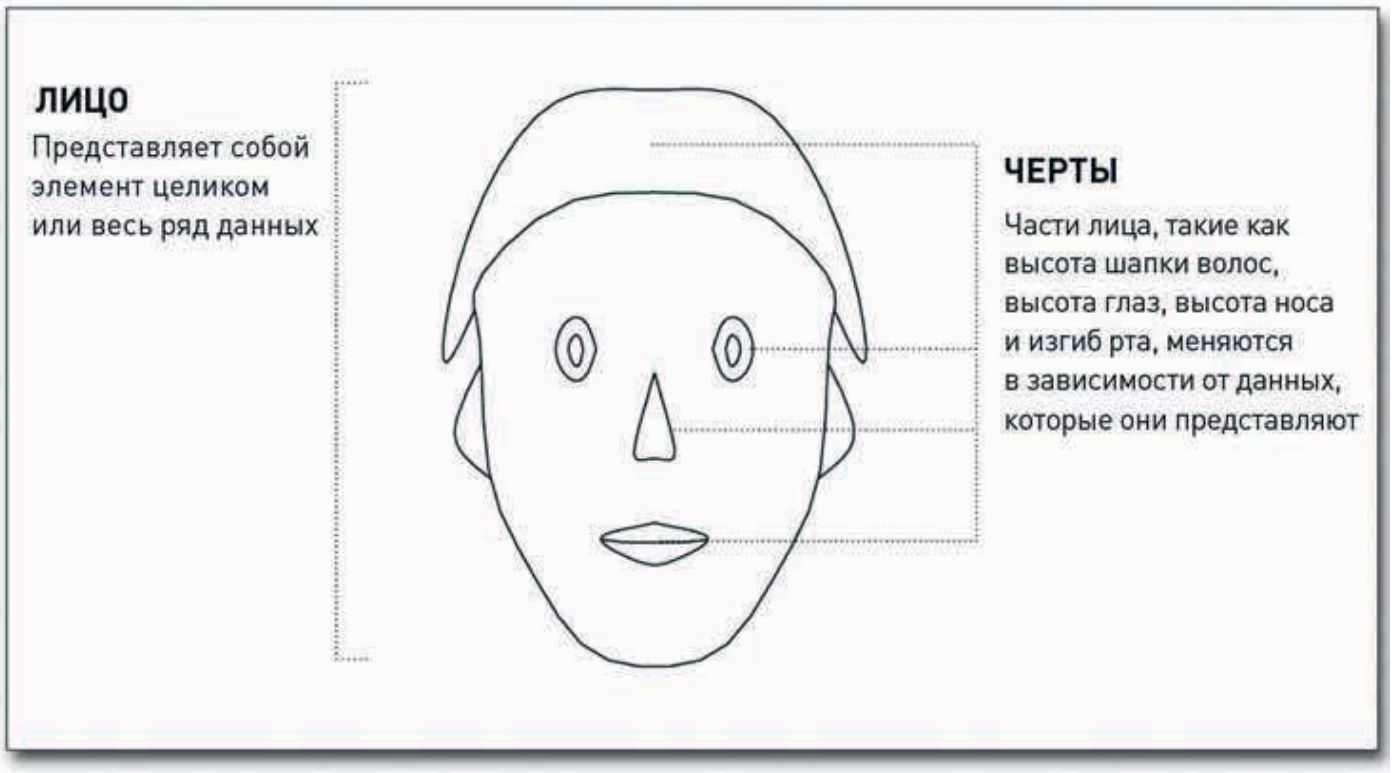
увлекательно. Если вы уже установили RColorBrewer, вам будет достаточно ввести следующий код, чтобы создать тепловую карту, используя голубую палитру с десятью оттенками:

```
library(RColorBrewer)
bball_heatmap <- heatmap(bball_matrix, Rowv=NA, Colv=NA, col = brewer.pal(9, "Blues"), scale="column", margins=c(5,10))
```



Задание 2. «Лица Чернова»

«Лица Чернова» — метод отображения многомерных данных в виде человеческого лица, придуманный математиком Германом Черновым и основанный на сильно развитой способности людей к восприятию лиц и мельчайших изменений в них. Смысл метода «лица Чернова» состоит в одновременной демонстрации множества переменных посредством специфического размещения частей человеческого лица, таких как уши, волосы, глаза и нос, на основе чисел определенного набора данных. Логический посыл таков: если вы в реальной жизни легко «прочитываете» лица людей, то вы сможете так же легко подметить даже мельчайшие изменения в них и тогда, когда их черты будут представлять некие данные. Предположение довольно смелое, но можно допустить, что верное. Большие значения принимают форму больших глаз или большой шапки волос, а меньшие значения делают черты лица мельче. Помимо размера, значение имеют и такие особенности, как изгиб рта или форма лица.

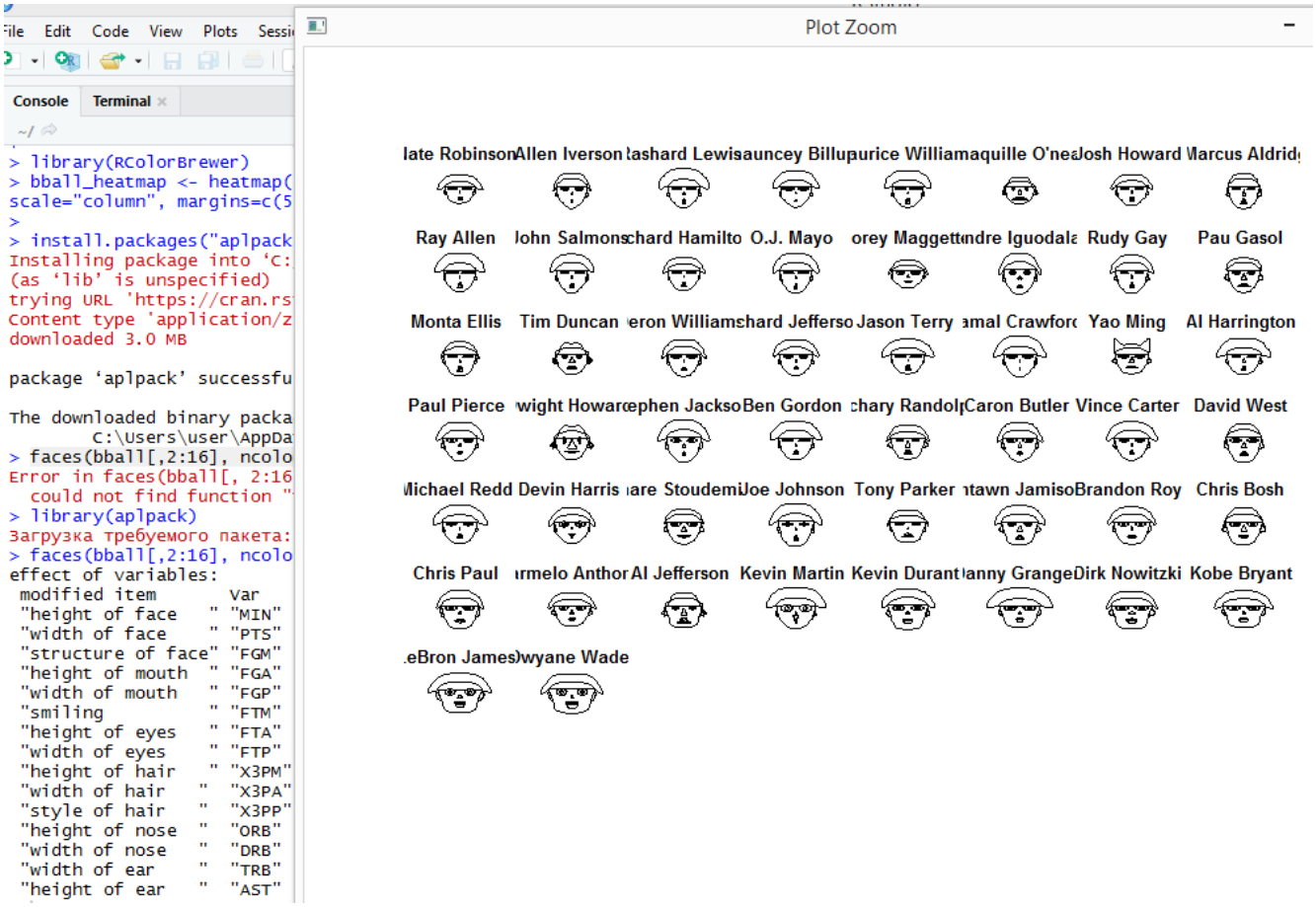


Структура диаграммы «Лица Чернова»

Давайте вернемся к данным, содержащим информацию о 50 лучших игроках НБА сезона 2008– 2009 годов. Нам нужно будет создать по одному лицу для каждого игрока. Не беспокойтесь — вам не придется вручную вычерчивать их все по очереди. В пакете aplpack в R есть функция faces(), которая поможет вам получить все необходимое. Если данный пакет у вас еще не установлен, сделайте это сейчас с помощью install.packages() или через Package Installer. Вы должны были еще раньше загрузить баскетбольные данные, когда создавали тепловую карту. Но если вы этого до сих пор не сделали, воспользуйтесь снова read.csv().

Приступим к созданию «лиц Чернова» с помощью функции faces(): faces(bball[,2:16], ncolors=0).

Пакет обычно загружается автоматически, но если загрузки не произошло и при попытке интерпретации кода выдается ошибка, тогда вам необходимо будет сделать это самостоятельно library(aplpack). Повторите команду: faces(bball[,2:16], ncolors=0).



У вашего набора данных есть 20 переменных плюс имена игроков. Однако функция faces(), предлагаемая aplpack, дает возможность работать не более чем с 15 переменными, потому что в ней есть только 15 черт лица, которые можно менять. Вот почему вы создадите подмножество из данных, содержащихся в колонках от 2 до 16.

Что будет представлять каждое лицо? Функция faces() меняет черты в следующем порядке, соответствующем порядку колонок данных:

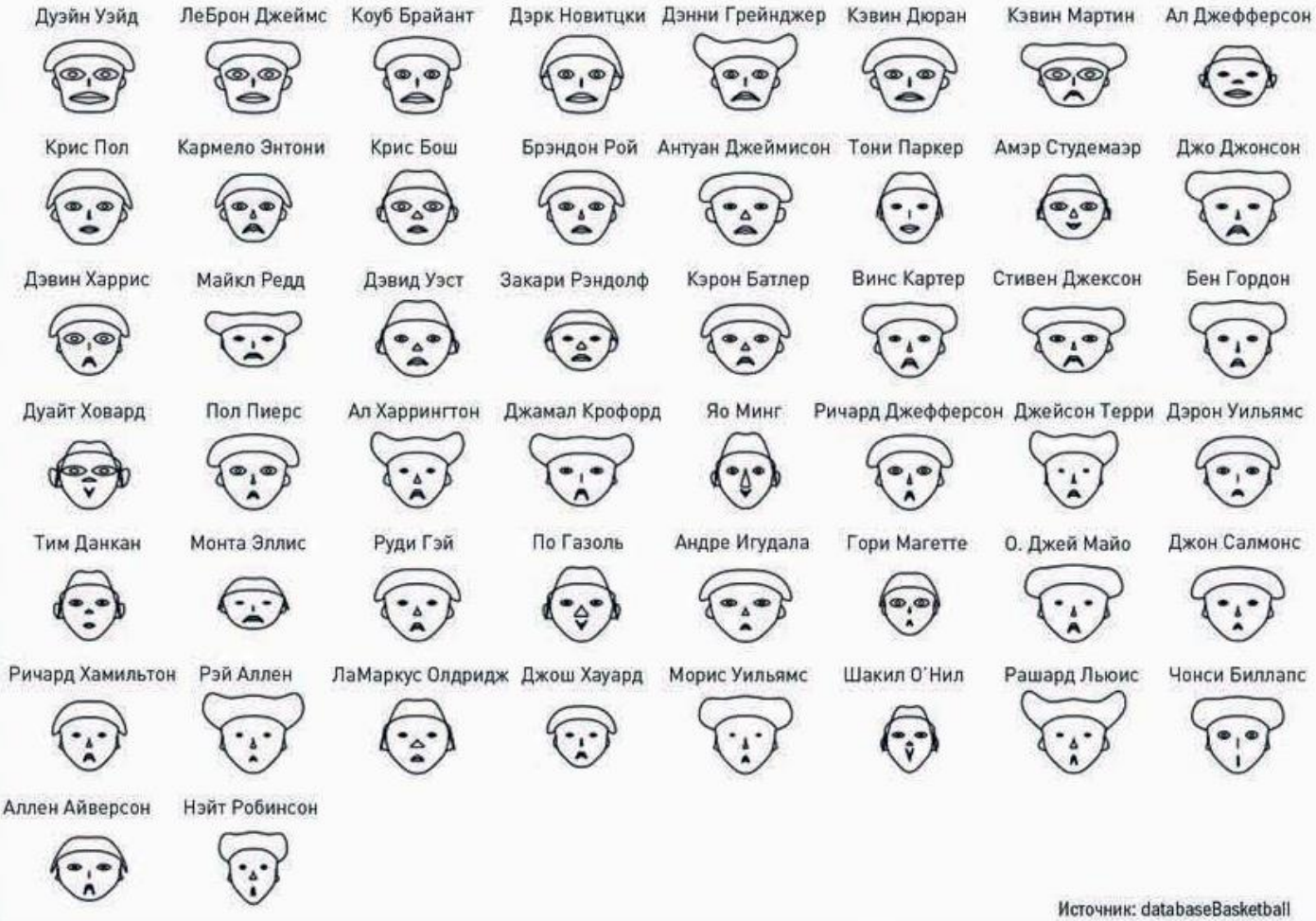
1. Высота лица.
2. Ширина лица.
3. Форма лица.
4. Высота рта.
5. Ширина рта.
6. Изгиб рта.
7. Высота глаз.
8. Ширина глаз.
9. Высота шапки волос.
10. Ширина волос.
11. Оформление волос.
12. Высота носа.
13. Ширина носа.
14. Ширина ушей.
15. Высота ушей.

В нашем примере высота лица будет представлять количество сыгранных игр, а высота рта — количество попаданий с игры в среднем. Чтобы сделать графику более читабельной, вы можете увеличить пробел между рядами и хотя бы коротко пояснить, что означают различные черты лица.

СРЕДНИЕ ПОКАЗАТЕЛИ ЗА ИГРУ В НБА

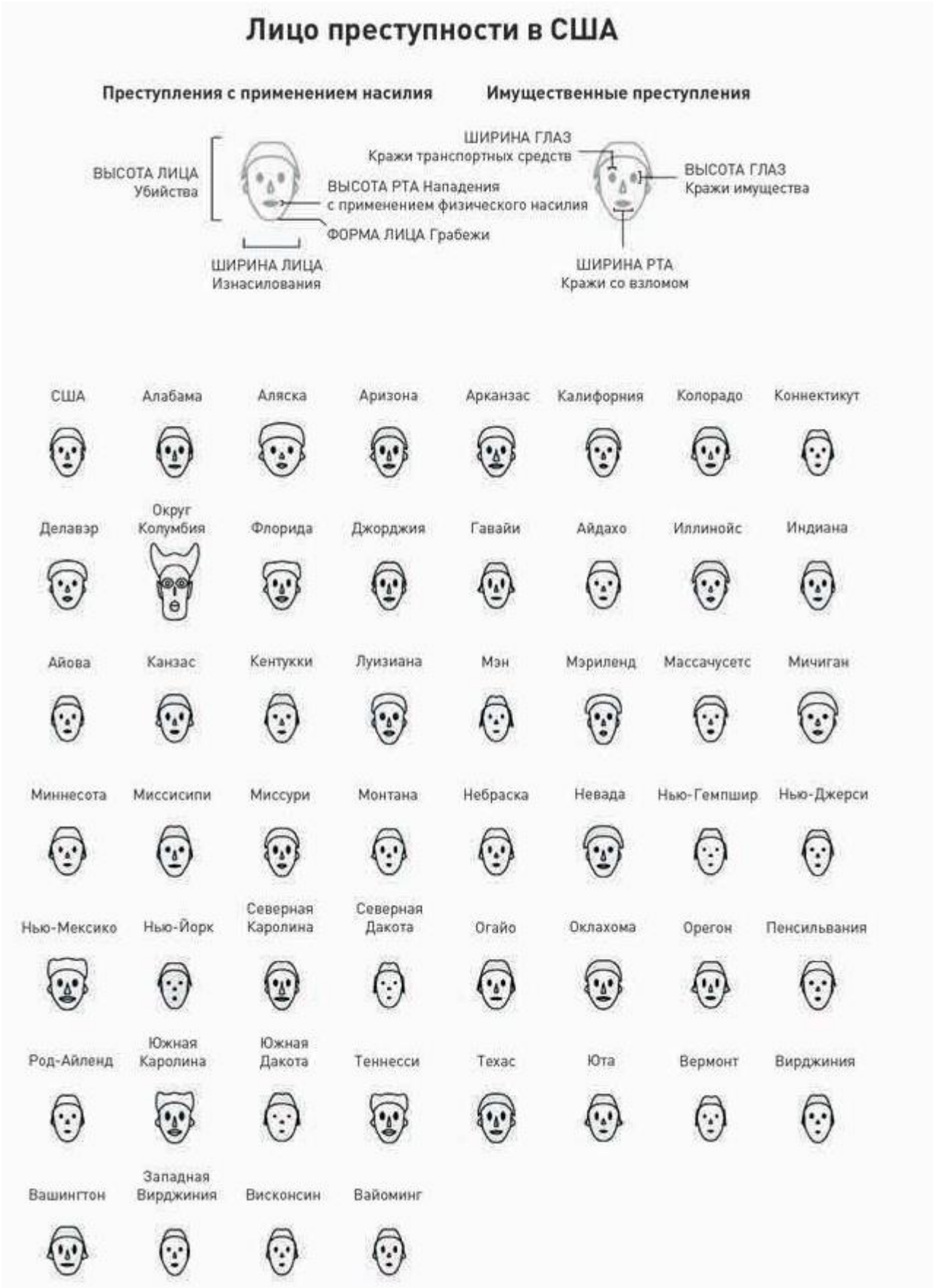
Мы использовали метод, известный как «лица Чернова», чтобы представить игровую статистику лучших баскетболистов сезона 2008–2009 гг. Это отнюдь не портреты настоящих игроков, идея в другом: с помощью отдельных черт лица представить определенные данные о каждом игроке. Спортсмены выстроены по количеству очков за игру.

Высота лица — количество игр	Изгиб рта — процент попаданий с игры	Форма шапки волос — неуспешные трехочковые броски
Ширина лица — минуты в игре	Высота глаз — свободные броски	Высота носа — подборы в нападении
Форма лица — очки за игру	Ширина глаз — попытки свободных бросков	Ширина носа — подборы в защите
Высота рта — попадания с игры	Высота шапки волос — процент свободных бросков	Ширина ушей — всего подборов мяча при отскоке
Ширина рта — попытки попадания с игры	Ширина шапки волос — трехочковые броски	Высота ушей — передачи



Источник: databaseBasketball

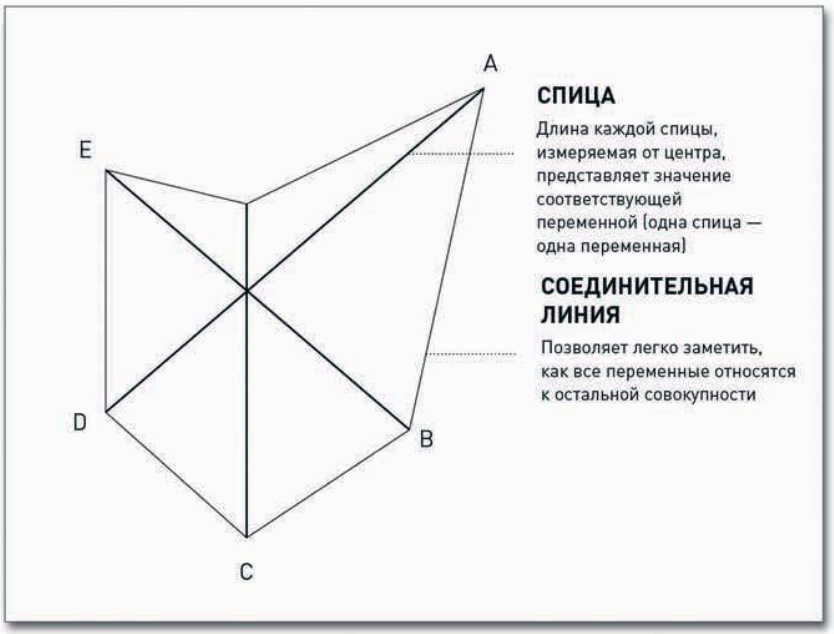
Задание 2**. Попробуйте получить следующую картинку, характеризующую преступность в США (задания 1-3).



Задание 3. Создание диаграммы-радара

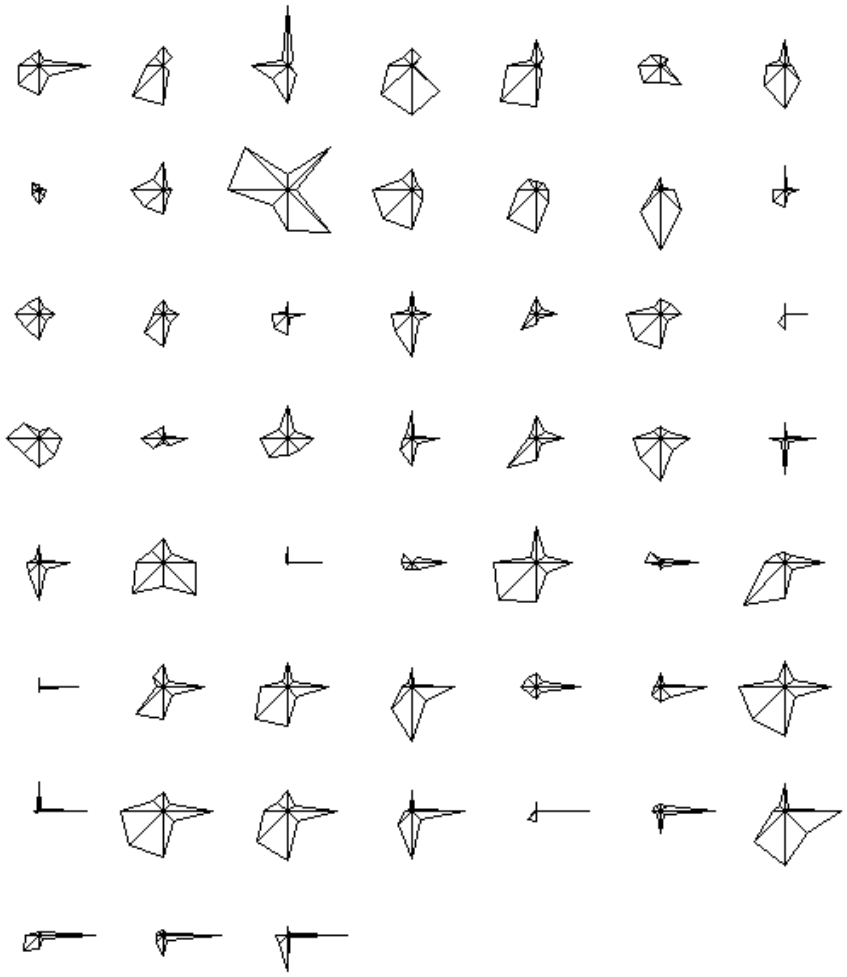
Для наглядного представления многомерных данных ту же самую идею можно применить, используя, однако, не лица, а другую графическую реализацию. Так, в зависимости от значений различных

показателей можно менять не черты лиц, а очертания фигур. В этом и состоит идея диаграмм-радаров, также известных как паутинные, или лепестковые, диаграммы (а еще их иногда называют диаграммами-звездами). Вы можете начертить несколько осей, по одной для каждой переменной, начинающихся с центра и расположенных на равном удалении от соседних, составляя таким образом круг («колесо»). Центр — это минимальное значение каждой из переменных, а конец оси — ее максимум. Если вы создаете диаграмму для одного элемента, начните с одной переменной и проведите соединительную линию до соответствующей точки на соседней оси. В конечном итоге вы получите нечто похожее на радар (или на паутину, на цветок, на звезду).



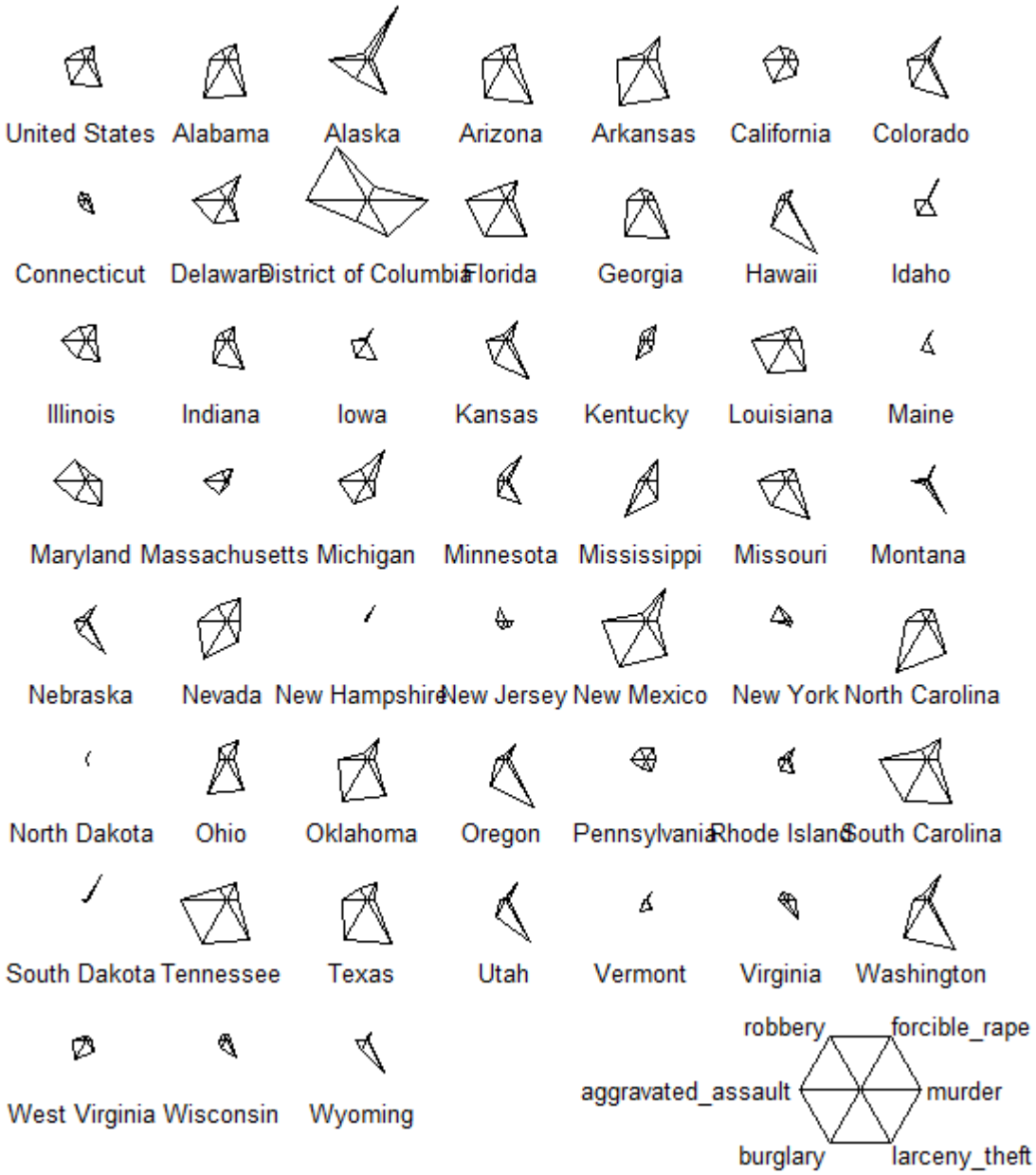
Для создания диаграммы-радара будем использовать все те же данные по преступности. Загрузим их в R еще раз.

После этого создать диаграмму-радар становится так же просто, как и нарисовать «лица Чернова». Воспользуйтесь функцией stars(), которая поставляется с базовым пакетом R: stars(crime). Получится что-то такое:



Помимо этого, вам необходимо дать также ключ к пониманию, что есть что. Здесь так же, как и с функцией faces(), соблюдается определенный порядок, но вы не знаете, где какая переменная и откуда все начинается. А потому позаботьтесь и о том, и о другом. Обратите внимание еще и вот на что: вы можете изменить первую колонку таким образом, чтобы в ней выстроился ряд названий штатов. Вы это уже делали, создавая тепловую карту. И вам нужно будет установить flip.labels на FALSE, так как вы не хотите, чтобы подписи имели разную высоту:

```
row.names(crime) <- crime$state
crime <- crime[,2:7]
stars(crime, flip.labels=FALSE, key.loc = c(15, 1.5))
```



Теперь уже легко заметить сходства и отличия. В варианте с лицами Чернова округ Колумбия, по сравнению с другими штатами, выглядел как клоун с выпученными глазами, а в варианте с диаграммами-радарами, как вы сами видите, он демонстрирует относительно высокие показатели по некоторым видам преступлений, но вместе с тем у него сравнительно низкие показатели по статьям «изнасилования» и «кражи со взломом». А еще легко можно выявить штаты с относительно невысоким уровнем преступности, такие, как Нью-Гемпшир и Род-Айленд. А есть и такие штаты, как Северная Каролина, у которых высоки показатели только по одному типу преступлений.

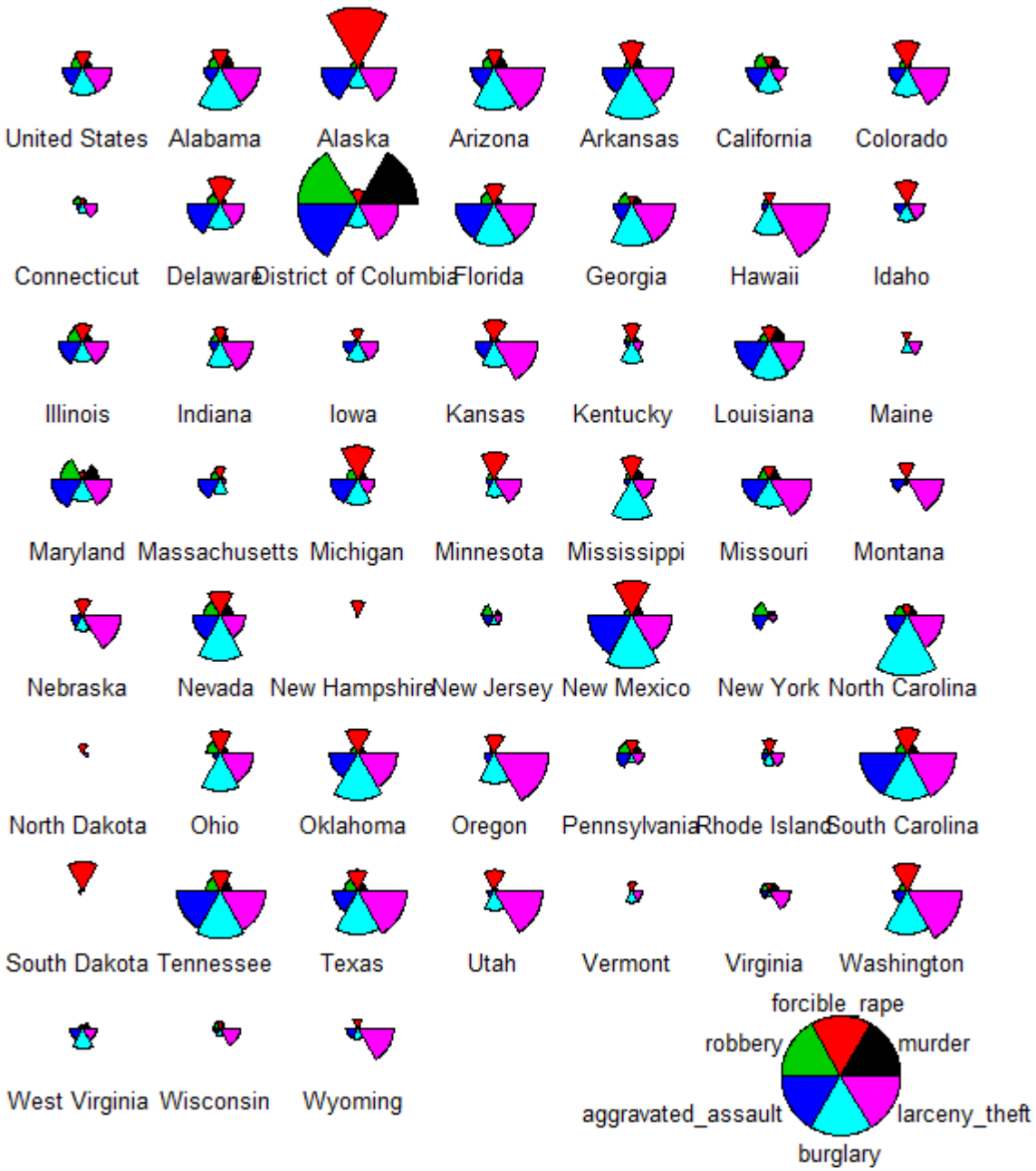
У этого метода визуализации данных есть еще две разновидности, которые, возможно, вы захотите опробовать применительно к вашим наборам данных. Первая разновидность ограничивает пространство размещения данных только верхней половиной круга:

```
stars(crime, flip.labels=FALSE, key.loc = c(15, 1.5), full=FALSE)
```



Вторая разновидность вместо размещения точек использует длину сегментов. На самом деле это уже не диаграмма-радар, а диаграмма Найтингейл (также известная как полярная диаграмма), но идея по сути такая же:

```
stars(crime, flip.labels=FALSE, key.loc = c(15, 1.5), draw.segments=TRUE)
```


United States
Alabama
Alaska
Arizona
Arkansas
California
Colorado
Connecticut
Delaware
District of Columbia
Florida
Georgia
Hawaii
Idaho
Illinois
Indiana
Iowa
Kansas
Kentucky
Louisiana
Maine
Maryland
Massachusetts
Michigan
Minnesota
Mississippi
Missouri
Montana
Nebraska
Nevada
New Hampshire
New Jersey
New Mexico
New York
North Carolina
North Dakota
Ohio
Oklahoma
Oregon
Pennsylvania
Rhode Island
South Carolina
South Dakota
Tennessee
Texas
Utah
Vermont
Virginia
Washington
West Virginia
Wisconsin
Wyoming
forcible_rape
robbery
murder
aggravated_assault
larceny_theft
burglary